



Universidad de Castilla-La Mancha
Escuela Superior de Ingeniería Informática de Albacete

Trabajo Fin de Grado
Grado en Ingeniería Informática
Tecnología específica de Computación

**Privacidad en Agregación de Rankings:
Aplicación en Aprendizaje Federado**

Celia Almendros Saelices

Junio, 2026

TRABAJO FIN DE GRADO

Grado en Ingeniería Informática

Tecnología específica de Computación

Privacidad en Agregación de Rankings: Aplicación en Aprendizaje Federado

Autor: Celia Almendros Saelices

Tutor: Juan Ángel Aledo Sánchez

Tutor: José Antonio Gámez Martín

Junio, 2026

Dedicado a mi familia, por creer siempre en mí.

Declaración de autoría

Yo, Celia Almendros Saelices, con DNI 06297407F, declaro que soy el único autor del trabajo fin de grado titulado “Privacidad en Agregación de Rankings: Aplicación en Aprendizaje Federado”, que el citado trabajo no infringe las leyes en vigor sobre propiedad intelectual, y que todo el material no original contenido en dicho trabajo está apropiadamente atribuido a sus legítimos autores.

Albacete, a 14 de junio de 2026

Fdo.: Celia Almendros Saelices

Declaración de uso de inteligencia artificial

En la elaboración de este Trabajo Fin de Grado se han utilizado herramientas de inteligencia artificial generativa de forma puntual como apoyo durante el proceso de desarrollo. Su uso ha estado orientado principalmente a mejorar la claridad de algunos fragmentos de la memoria, revisar la redacción técnica, contrastar dudas conceptuales y apoyar la depuración de partes concretas del código.

Estas herramientas se han empleado únicamente como recurso auxiliar. Las decisiones metodológicas, el diseño de los experimentos, la implementación final, la ejecución de las pruebas, la interpretación de los resultados y la redacción definitiva han sido realizadas por la autora. En ningún caso estas herramientas han sustituido el análisis propio ni la toma de decisiones del trabajo. La autora asume completamente la responsabilidad del contenido del manuscrito presentado.

Resumen

Privacidad en agregación de rankings: aplicación en aprendizaje federado

El Problema de Agregación de Rankings (RAP) aparece en escenarios de decisión colectiva, sistemas de recomendación y ciencia de datos, donde se desea obtener una ordenación agregada a partir de varias preferencias de entrada. En entornos reales, estas preferencias pueden contener empates o información incompleta. Por ello, este Trabajo de Fin de Grado estudia la agregación mediante el *Optimal Bucket Order Problem* (OBOP), cuya salida es un *bucket order*: una ordenación formada por grupos de alternativas empatadas.

El trabajo parte de una implementación exacta del OBOP mediante Programación Lineal Entera (ILP). Sobre esta base se integran distintas estrategias de introducción de ruido para proteger parcialmente la información individual contenida en los rankings y estudiar el compromiso entre introducción de ruido y calidad del resultado. Para ello, se diseñan técnicas que actúan en diferentes fases del flujo de agregación: directamente sobre la matriz de precedencias, sobre los rankings individuales y sobre puntuaciones (*scores*) latentes.

Además, se desarrolla una simulación federada en la que los rankings se reparten entre varios clientes. Cada cliente trabaja con su información local y construye sus matrices intermedias de forma aislada, mientras que el servidor combina la información recibida para obtener el *bucket order* final. Este planteamiento permite estudiar la agregación cuando los rankings originales no se centralizan directamente.

La evaluación experimental se realiza sobre instancias reales de rankings y compara el enfoque centralizado con el federado. El análisis se centra en la estabilidad del *bucket order* ante la introducción de ruido y en el equilibrio entre la protección de la información individual y la conservación de la calidad del resultado.

Abstract

Privacy in rank aggregation: application in federated learning

The Ranking Aggregation Problem (RAP) appears in collective decision-making, recommendation systems, and data science scenarios, where the goal is to obtain an aggregated ranking from several input preferences. In real-world environments, these preferences may contain ties or incomplete information. Therefore, this work studies aggregation through the *Optimal Bucket Order Problem* (OBOP), whose output is a *bucket order*: a ranking formed by groups of tied alternatives.

This work is based on an exact implementation of the OBOP using Integer Linear Programming (ILP). Based on this, different noise introduction strategies are integrated to partially protect the individual information contained in the rankings and to study the trade-off between noise introduction and the quality of the result. To this end, techniques are designed to act at different phases of the aggregation workflow: directly on the precedence matrix, on the individual rankings, and on latent *scores*.

In addition, a federated simulation is developed in which the rankings are distributed among several clients. Each client works with its local information and builds its intermediate matrices independently, while the server combines the received information to obtain the final *bucket order*. This approach allows for the study of aggregation when the original rankings are not directly centralized.

The experimental evaluation is carried out on real-world ranking instances and compares the centralized approach with the federated one. The analysis focuses on the stability of the *bucket order* under noise introduction and on the balance between protecting individual information and preserving the quality of the result.

Agradecimientos

En primer lugar, a mis padres, por ser mi mayor apoyo, acompañarme en cada paso y celebrar mis logros como si fueran propios. A mis hermanas, Esther y María: a Esther, por recorrer este camino conmigo y compartir cada etapa de esta experiencia, y a María, por animarme y sacarme siempre una sonrisa. Si he llegado hasta aquí, es también, en gran parte, gracias a vosotros.

También quiero agradecer a todos los profesores que me han acompañado durante la carrera, por todo lo que me han enseñado y por su contribución a mi formación. En especial, a mis tutores, Juan Ángel y José Antonio, por confiar en mí desde el principio y guiarme durante el desarrollo de este trabajo. Gracias por vuestra disponibilidad, paciencia e implicación a lo largo de todo el proceso.

A mis compañeros y amigos, gracias por compartir conmigo esta etapa. Gracias por vuestro apoyo, vuestra ayuda y todos los momentos que hemos vivido juntos. Especialmente a Alex, por ser mi compañero de equipo y por impulsarme siempre a mejorar y aprender. Sin vosotros, estos años no habrían sido los mismos.

A todas las personas que me han acompañado durante esta etapa, gracias por hacer de este camino algo mucho más especial.

Índice general

Resumen	ix
Abstract.....	xi
Agradecimientos	xiii
Índice de figuras	xix
Índice de tablas	xxi
Índice de algoritmos.....	xxiii
1 Introducción	1
1.1 Motivación	3
1.2 Objetivos y competencias	3
1.3 Estructura del documento.....	5
2 Estado del arte.....	7
2.1 Problema de Agregación de Rankings (RAP)	7
2.1.1 <i>Tipologías de rankings</i>	8
2.1.2 <i>Funciones de distancia</i>	9
2.2 Optimal Bucket Order Problem (OBOP).....	10
2.2.1 <i>Estructura de entrada</i>	10
2.2.2 <i>Estructura de salida</i>	11
2.2.3 <i>Función objetivo</i>	11
2.2.4 <i>Complejidad</i>	12

2.3	Métodos de resolución del OBOP	12
2.3.1	<i>Algoritmos heurísticos y metaheurísticos</i>	13
2.3.2	<i>Formulación de Programación Lineal Entera del OBOP</i>	14
2.4	Privacidad en agregación de rankings	16
2.5	Aprendizaje Federado	17
3	Metodología experimental	21
3.1	Estrategias de introducción de ruido	22
3.1.1	<i>Ruido directo sobre la matriz</i>	22
3.1.2	<i>Ruido directo sobre rankings individuales</i>	26
3.1.3	<i>Ruido sobre scores latentes y reconstrucción de la matriz</i>	30
3.2	Procedimiento centralizado con ruido	33
3.3	Simulación federada	34
3.4	Métricas de evaluación	37
3.4.1	<i>Coefficiente de correlación de Kendall τ_b</i>	37
3.4.2	<i>Pérdida de calidad respecto a la matriz original</i>	37
3.4.3	<i>Distancia de entrada, distancia de salida y sensibilidad</i>	38
3.4.4	<i>Número de buckets</i>	40
3.4.5	<i>Métricas en la simulación federada</i>	40
3.5	Metodología seguida durante el desarrollo del trabajo	41
4	Experimentación y análisis de resultados	43
4.1	Diseño experimental	43
4.1.1	<i>Conjuntos de datos</i>	44
4.2	Baseline centralizado sin ruido	45
4.3	Experimentos centralizados con ruido	46
4.3.1	<i>Efecto del parámetro de ruido b</i>	46
4.3.2	<i>Análisis detallado de las instancias</i>	48
4.3.3	<i>Comparación global de las estrategias centralizadas</i>	57
4.4	Resultados de la simulación federada	59
4.4.1	<i>Simulación federada sin ruido</i>	60
4.4.2	<i>Comparación entre centralizado y federado</i>	60
5	Conclusiones y líneas futuras	65
5.1	Conclusiones	65
5.2	Trabajos futuros	68
5.3	Justificación de las competencias adquiridas	69

Bibliografía	74
A Detalles de implementación	77
A.1 Organización general del proyecto	77
A.2 Módulos principales	78
A.3 Lectura de datos y construcción de la matriz de precedencias	79
A.4 Resolución del OBOP	80
A.5 Estrategias de introducción de ruido	80
A.6 Implementación de la simulación federada	81
A.7 Métricas y resultados	82
A.8 Ejecución de los experimentos	83
A.9 Dependencias	83
B Resultados complementarios de los experimentos con ruido	85
B.1 Instancia 00035-00000006.soc	85
B.2 Instancia 00068-00000301.soi	86
B.3 Instancia 00071-00000033.toi	87
B.4 Métricas locales complementarias de la simulación federada	88
C Recursos software y hardware	89
C.1 Recursos software	89
C.2 Recursos hardware	90

Índice de figuras

2.1	Esquema general del aprendizaje federado.	18
2.2	Esquema de agregación federada aplicado al OBOP.	20
3.1	Función de densidad de la distribución de Laplace.	23
4.1	Efecto del parámetro b sobre la distancia de entrada.	47
4.2	Relación entre el nivel de ruido y las métricas de evaluación.	58
A.1	Diagrama de clases de la simulación federada.	82

Índice de tablas

4.1	Colecciones e instancias seleccionadas.	45
4.2	Resultados del baseline centralizado.	46
4.3	Instancia 00006-00000040.toc. Ruido sobre matriz.	49
4.4	Instancia 00006-00000040.toc. Ruido sobre rankings.	50
4.5	Instancia 00006-00000040.toc. Ruido sobre <i>scores</i>	51
4.6	Instancia 00035-00000006.soc. Ruido sobre matriz.	53
4.7	Instancia 00035-00000006.soc. Ruido sobre <i>scores</i>	54
4.8	Instancia 00068-00000301.soi. Ruido sobre rankings.	55
4.9	Instancia 00071-00000033.toi. Ruido sobre matriz.	56
4.10	Instancia 00071-00000033.toi. Ruido sobre <i>scores</i>	57
4.11	Simulación federada sin ruido.	60
4.12	Simulación federada: Métricas globales.	61
4.13	Simulación federada: influencia del número de clientes.	62
A.1	Módulos principales de la implementación.	79
B.1	Instancia 00035-00000006.soc. Ruido sobre rankings.	85
B.2	Instancia 00068-00000301.soi. Ruido sobre matriz.	86
B.3	Instancia 00068-00000301.soi. Ruido sobre <i>scores</i>	87
B.4	Instancia 00071-00000033.toi. Ruido sobre rankings.	87
B.5	Simulación federada: Métricas locales.	88
C.1	Recursos hardware del sistema.	90

Índice de algoritmos

3.1	Introducción de ruido sobre la matriz.	26
3.2	Introducción de ruido sobre rankings individuales.	30
3.3	Introducción de ruido sobre <i>scores</i> latentes.	33
3.4	Procedimiento experimental centralizado.	34
3.5	Procedimiento de simulación federada.	36

1. Introducción

La *agregación de rankings* surge en contextos en los que se dispone de varias ordenaciones sobre un mismo conjunto de alternativas y se desea obtener, a partir de ellas, un resultado común que sintetice la información disponible. Estas ordenaciones pueden proceder de procesos de decisión colectiva, sistemas de recomendación, motores de búsqueda o evaluaciones de expertos, entre otros escenarios. Aunque cada ranking refleja una valoración particular, el objetivo es construir un consenso que recoja de forma fiel e interpretable la preferencia conjunta [1, 2].

En muchas situaciones reales, las preferencias no se expresan necesariamente como ordenaciones estrictas. Es frecuente que existan alternativas consideradas equivalentes o entre las que no se puede establecer una preferencia clara, dando lugar a rankings con empates. Este tipo de estructuras, conocidas como *bucket orders*, permiten agrupar alternativas empatadas dentro de un mismo bloque, manteniendo a su vez un orden entre los distintos bloques o *buckets* [2, 3, 4].

Dentro de este contexto, el *Optimal Bucket Order Problem* (OBOP) proporciona un marco formal para obtener un consenso en forma de *bucket order* a partir de una matriz de precedencias C . El objetivo consiste en encontrar el ranking con empates cuya matriz asociada sea lo más cercana posible a la información (matriz) de entrada [2, 4]. Para resolver este problema de forma exacta, se ha propuesto una formulación mediante *Programación Lineal Entera* (ILP) que permite obtener soluciones óptimas en instancias de tamaño reducido o moderado [5].

Además de la calidad del consenso, un aspecto cada vez más relevante en la agregación de rankings es la protección de la información original. Un ranking individual puede revelar información sensible sobre quien lo genera, como gustos personales, afinidades políticas, valoraciones profesionales o decisiones estratégicas. Por ello, resulta

de interés estudiar mecanismos que permitan reducir la exposición de estos datos sin degradar en exceso la utilidad del resultado agregado. En este trabajo, esta idea se aborda mediante la introducción de ruido sobre distintas representaciones de las preferencias, con el objetivo de dificultar la recuperación exacta de la información inicial y analizar cómo afecta dicha perturbación al *bucket order* final.

En este contexto, la privacidad no solo depende del ruido introducido sobre los datos, sino también de la forma en que estos se recopilan y procesan. Cuando todos los rankings se centralizan en un único punto, aumenta la exposición de las preferencias originales. Por ello, en este trabajo se considera también una aproximación federada sencilla, en la que la agregación se estudia bajo un escenario distribuido. Esta variante permite analizar si es posible obtener un *bucket order* similar al centralizado reduciendo, al mismo tiempo, la necesidad de reunir directamente todos los rankings originales.

A partir de estas ideas, este Trabajo de Fin de Grado se centra en el estudio experimental de estrategias de perturbación aplicadas a problemas de agregación de rankings con empates. El propósito es analizar cómo distintas formas de introducir ruido en los datos afectan al *bucket order* óptimo obtenido en el OBOP. Para ello, se parte de rankings de entrada, se construye la matriz de precedencias correspondiente y se resuelve el problema mediante ILP para obtener un *bucket order* de referencia. Posteriormente, se aplican distintas perturbaciones sobre los datos y se estudia cómo varía el *bucket order* resultante, prestando especial atención al compromiso entre la modificación de los datos y la conservación de la calidad.

De forma complementaria, se incluye una simulación federada del proceso de agregación con introducción de ruido local. En ella, los rankings se dividen entre varios clientes, que trabajan con su información local sin compartir directamente las preferencias originales [6]. Antes de realizar la agregación global, se introduce ruido localmente, de modo que la información enviada al servidor ya se encuentra modificada [7]. Este planteamiento permite comparar el enfoque centralizado con una aproximación distribuida y analizar cómo se ve afectado el *bucket order* obtenido cuando el ruido se aplica sobre las partes locales.

Como parte del proyecto se ha desarrollado un repositorio público en GitHub¹ que recoge el código fuente, los scripts de ejecución y los resultados generados durante la experimentación. Este repositorio permite reproducir el entorno experimental implementado y facilita su ampliación futura.

¹https://github.com/celiaalmendross/agregacion_rankings_TFG

1.1. Motivación

La motivación principal de este trabajo es disponer de un marco experimental que permita estudiar, de forma controlada, cómo afecta el ruido a la agregación de rankings con empates. La introducción de perturbaciones se plantea como una capa adicional de privacidad, ya que modifica la información original con el objetivo de dificultar que pueda reconstruirse de forma exacta el ranking aportado por cada individuo. Sin embargo, no todas las estrategias de ruido tienen por qué afectar del mismo modo al resultado final ni ofrecer el mismo equilibrio entre protección de la información individual y conservación de la calidad del *bucket order* obtenido.

En el caso del OBOP, esta cuestión resulta especialmente relevante porque el consenso obtenido no es una ordenación estricta, sino un *bucket order*. Por tanto, el ruido puede modificar no solo las relaciones de precedencia entre alternativas, sino también la estructura de empates del *bucket order*, rompiendo empates existentes, creando nuevos grupos de indiferencia o desplazando alternativas entre *buckets*.

Por este motivo, el trabajo se plantea desde una perspectiva experimental: implementar distintas estrategias de perturbación, resolver el OBOP antes y después de introducir ruido, y comparar los *bucket orders* obtenidos. La formulación exacta mediante ILP permite disponer de una solución óptima de referencia sobre la que evaluar con precisión la pérdida de calidad de la solución obtenida por cada estrategia [5].

1.2. Objetivos y competencias

El objetivo principal de este Trabajo de Fin de Grado es diseñar e implementar un entorno experimental para estudiar el efecto de distintas estrategias de introducción de ruido en problemas de agregación de rankings con empates, tomando como referencia el *bucket order* óptimo obtenido en el *Optimal Bucket Order Problem* (OBOP). Además, se incorpora una simulación federada sencilla para analizar cómo se comporta la agregación cuando la perturbación se aplica localmente sobre datos distribuidos entre varios clientes.

Este objetivo general se descompone en los siguientes subobjetivos:

- Estudiar el problema de agregación de rankings con empates, así como los conceptos necesarios para representar consensos en forma de *bucket order*.

-
- Analizar el OBOP como marco formal para obtener un *bucket order* óptimo a partir de una matriz de precedencias.
 - Resolver el OBOP mediante una formulación exacta de Programación Lineal Entera, obteniendo un *bucket order* óptimo de referencia.
 - Diseñar e implementar distintas estrategias de introducción de ruido sobre los datos utilizados en la agregación.
 - Evaluar el efecto del ruido sobre el *bucket order* obtenido, comparando la solución original con las soluciones perturbadas mediante métricas de similitud y pérdida de calidad.
 - Implementar una simulación federada sencilla del proceso de agregación de rankings con introducción de ruido local.
 - Comparar experimentalmente el enfoque centralizado con la aproximación federada propuesta.
 - Analizar experimentalmente el compromiso entre el nivel de ruido introducido y la conservación de la calidad del *bucket order*.

En cuanto a las principales competencias de la Tecnología Específica de Computación que se desarrollarán en este Trabajo de Fin de Grado para alcanzar los objetivos anteriormente descritos, serán las siguientes:

- [CM3] Capacidad para evaluar la complejidad computacional de un problema, conocer estrategias algorítmicas que puedan conducir a su resolución y recomendar, desarrollar e implementar aquella que garantice el mejor rendimiento de acuerdo con los requisitos establecidos.
- [CM4] Capacidad para conocer los fundamentos, paradigmas y técnicas propias de los sistemas inteligentes y analizar, diseñar y construir sistemas, servicios y aplicaciones informáticas que utilicen dichas técnicas en cualquier ámbito de aplicación.
- [CM7] Capacidad para conocer y desarrollar técnicas de aprendizaje computacional y diseñar e implementar aplicaciones y sistemas que las utilicen, incluyendo las dedicadas a extracción automática de información y conocimiento a partir de grandes volúmenes de datos.

1.3. Estructura del documento

El resto de la memoria se organiza en cuatro capítulos. En el Capítulo 2 se presentan los conceptos teóricos sobre los que se apoya el trabajo. En primer lugar, se introduce el problema de agregación de rankings (RAP), posteriormente, se describe el Optimal Bucket Order Problem (OBOP). Además, se revisan los conceptos de privacidad y aprendizaje federado que motivan la parte experimental del proyecto.

En el Capítulo 3 se describe la metodología experimental seguida. Se presentan las estrategias de introducción de ruido consideradas, el procedimiento centralizado, la simulación federada basada en matrices locales y las métricas utilizadas para evaluar la estabilidad y la calidad del *bucket order* obtenido.

En el Capítulo 4 se recogen los experimentos realizados y el análisis de resultados. Primero se estudia el escenario centralizado, comparando el *bucket order* original con los obtenidos tras introducir ruido. Después se analiza la simulación federada, centrándose en la comparación entre el *bucket order* federado y el resultado obtenido en el escenario centralizado.

Finalmente, el Capítulo 5 recoge las principales conclusiones del trabajo, las limitaciones encontradas y algunas líneas futuras para ampliar el estudio realizado.

2. Estado del arte

El presente capítulo reúne los conceptos clave sobre los que se construye este Trabajo de Fin de Grado. En primer lugar, se introduce el Problema de Agregación de Rankings (RAP), con especial atención a preferencias que admiten *empates* entre alternativas. A continuación, se presenta el *Optimal Bucket Order Problem* (OBOP) como marco específico para obtener un consenso en forma de *bucket order*, junto con los principales métodos propuestos para su resolución, incluyendo algoritmos heurísticos y la formulación exacta mediante Programación Lineal Entera. Finalmente, se contextualiza la privacidad en agregación de rankings, el uso de ruido como mecanismo para proteger las preferencias individuales y el *Aprendizaje Federado* como paradigma para tratar escenarios donde la información se encuentra distribuida entre varios clientes.

2.1. Problema de Agregación de Rankings (RAP)

El *Problema de Agregación de Rankings* (RAP, *Rank Aggregation Problem*) consiste en obtener, a partir de un conjunto de rankings de entrada definidos sobre un dominio común de alternativas, un ranking de consenso que sintetice dicha información y actúe como representante del conjunto. En este marco, la agregación puede interpretarse como un operador de fusión de preferencias ordinales procedentes de fuentes diversas, las cuales pueden reflejar criterios o puntos de vista distintos. Asimismo, el RAP se ha descrito como un término *paraguas* que engloba distintas variantes, tanto por la estructura de los datos de entrada y salida como por la función objetivo empleada para evaluar la calidad del resultado [1, 2, 4, 7].

Formalmente, dado un conjunto de rankings $R = \{R_1, \dots, R_m\}$ sobre n alternativas, se busca un ranking agregado π_0 que minimice la discrepancia total respecto a los

rankings de entrada. De forma general, este planteamiento puede expresarse como:

$$\pi_0 = \arg \min_{\pi \in \Omega} \sum_{i=1}^m d(R_i, \pi), \quad (2.1)$$

donde π denota un ranking candidato y π_0 es el consenso seleccionado como el que alcanza el menor valor de la expresión anterior [1, 8]. El conjunto Ω representa el espacio de soluciones permitido, mientras que $d(R_i, \pi)$ cuantifica la diferencia entre el ranking de entrada R_i y el candidato π [1, 8].

2.1.1. Tipologías de rankings

La estructura de los datos de entrada y salida puede variar, condicionando el dominio de soluciones y las propiedades del consenso [1, 3]. Se distinguen principalmente los siguientes tipos de rankings:

- **Rankings completos:** incluyen todas las alternativas del conjunto [1, 2].
- **Rankings parciales (o incompletos):** solo ordenan un subconjunto de alternativas o dejan relaciones sin especificar [2, 3].
- **Rankings sin empates:** establecen una preferencia estricta entre las alternativas consideradas, de forma que no existen dos alternativas situadas en la misma posición.
- **Rankings con empates:** permiten asignar la misma posición a dos o más alternativas cuando no se establece una preferencia estricta entre ellas, manteniendo un orden entre los distintos grupos de alternativas empatadas [2, 3].

Este estudio se centra en el caso de rankings completos con empates, conocidos como *bucket orders*. En este tipo de rankings, todas las alternativas del conjunto aparecen en la ordenación, aunque algunas de ellas pueden quedar agrupadas en un mismo nivel de preferencia. Formalmente, un *bucket order* se define como una *partición ordenada* del conjunto de n alternativas en k subconjuntos disjuntos no vacíos $B_1, \dots, B_k$ (con $1 \leq k \leq n$), denominados *buckets* [2, 4]. Esta representación establece dos condiciones:

- (i) Las alternativas dentro de un mismo *bucket* B_i se consideran empatadas [2, 4].
- (ii) Si $i < j$, entonces cualquier alternativa de B_i precede a cualquier alternativa de B_j [2, 4].

Para describir un *bucket order* de forma compacta, es habitual separar con comas las alternativas de un mismo *bucket* y utilizar el símbolo $|$ para separar *buckets* consecutivos. Por ejemplo, $a, b | c | d, e$ indica que a y b están empatados en la primera posición, preceden a c , y este precede a d y e , que están empatados. Esta estructura se retomará en el siguiente apartado como base para formular el problema de consenso orientado a empates (OBOP) [2, 4].

2.1.2. Funciones de distancia

La función de distancia d es un componente central de cada instancia del RAP, ya que define cómo se cuantifica la diferencia entre rankings y, en consecuencia, cómo se evalúa la calidad del consenso [2, 9]. Algunas de las funciones de distancia más utilizadas incluyen:

- **Distancia de Kendall–Tau:** cuenta el número total de desacuerdos por pares entre dos rankings. Dados dos rankings σ y π , se expresa como:

$$K(\sigma, \pi) = |\{(i, j) : i < j, (\sigma(i) < \sigma(j) \wedge \pi(i) > \pi(j)) \vee (\sigma(i) > \sigma(j) \wedge \pi(i) < \pi(j))\}|.$$

Esta expresión contabiliza aquellos pares de alternativas que aparecen en un orden en el ranking σ y en el orden contrario en el ranking π . El problema que minimiza esta distancia se conoce como *Agregación Óptima de Kemeny* [7, 10].

- **Distancia de Spearman’s Footrule:** mide la discrepancia a partir de posiciones, calculando la suma de las diferencias absolutas entre la posición de cada alternativa en ambos rankings [11].
- **Distancia de Cayley:** cuantifica el número mínimo de transposiciones requeridas para transformar un ranking en otro [9, 11].
- **Distancias para estructuras complejas y generalizaciones:** existen extensiones de las distancias clásicas para adaptarlas a rankings con empates o rankings parciales, así como generalizaciones que incorporan pesos o penalizaciones específicas [3, 9, 11].

La elección de la función de distancia d , junto con la estructura del ranking considerada (completo, parcial, con o sin empates), determina variantes concretas del RAP [2, 9]. En muchos de estos casos, el cálculo del consenso resulta computacionalmente difícil (*NP-hard*), lo que ha motivado el desarrollo de algoritmos exactos para instan-

cias de tamaño moderado y, de forma más habitual, métodos aproximados y heurísticos para escenarios de mayor escala [3, 8, 10].

2.2. Optimal Bucket Order Problem (OBOP)

El **Optimal Bucket Order Problem** o Problema de Ordenamiento Óptimo con Empates (OBOP) es una variante del problema de agregación de rankings en la que el consenso se busca en el espacio de rankings completos con empates, denominados *bucket orders* [2, 4]. A diferencia de formulaciones que operan directamente sobre los rankings individuales, el OBOP toma como entrada una matriz de preferencias por pares, que habitualmente resume las preferencias de un conjunto de usuarios, lo que permite expresar el problema en términos matriciales [2, 4].

2.2.1. Estructura de entrada

La entrada del OBOP se representa mediante una matriz cuadrada $C \in [0, 1]^{n \times n}$, denominada *pair order matrix* o *matriz de precedencias* [2, 4]. Para cada par de alternativas (u, v) , el valor $C(u, v)$ representa la preferencia de u sobre v según el conjunto de rankings de entrada y puede interpretarse como la proporción de veces, o probabilidad estimada, con la que u aparece por delante de v [4]. Por definición, en esta representación se cumple $C(u, u) = 0.5$ y $C(u, v) + C(v, u) = 1$ para todo par de alternativas [2, 4].

Ejemplo de construcción de la matriz C . Supongamos un conjunto de cuatro alternativas $\{a, b, c, d\}$ y cuatro rankings de entrada:

$$R_1 = a|b|c|d, \quad R_2 = b|a|d|c, \quad R_3 = a|c|b|d, \quad R_4 = c|a|b|d.$$

En este ejemplo, la matriz C se construye mediante proporciones. Para cada par de alternativas (u, v) , se calcula la proporción de rankings en los que u precede a v . Por ejemplo, a precede a b en tres de los cuatro rankings, por lo que $C(a, b) = 3/4$ y

$C(b, a) = 1/4$. Aplicando este criterio a todos los pares se obtiene:

$$C = \begin{pmatrix} 0.5 & 0.75 & 0.75 & 1 \\ 0.25 & 0.5 & 0.5 & 1 \\ 0.25 & 0.5 & 0.5 & 0.75 \\ 0 & 0 & 0.25 & 0.5 \end{pmatrix}.$$

2.2.2. Estructura de salida

El OBOP devuelve una solución óptima en forma de *bucket order*, que denotaremos por β [2, 4]. Para comparar dicho consenso con la matriz de entrada C , el *bucket order* se representa mediante una matriz $B_\beta \in \{0, 0.5, 1\}^{n \times n}$, habitualmente denominada *bucket order matrix* [2, 4]:

$$B_\beta(u, v) = \begin{cases} 1, & \text{si } u \text{ precede a } v \text{ en el } \textit{bucket order } \beta, \\ 0, & \text{si } v \text{ precede a } u \text{ en el } \textit{bucket order } \beta, \\ 0.5, & \text{si } u \text{ y } v \text{ están empatados en el } \textit{bucket order } \beta. \end{cases}$$

Además de la regla de asignación anterior, la *bucket order matrix* B_β satisface las siguientes propiedades [2, 4]:

- (i) **Diagonal:** $B_\beta(u, u) = 0.5$ para cualquier u .
- (ii) **Complementariedad:** $B_\beta(u, v) + B_\beta(v, u) = 1$.
- (iii) **Transitividad:** si $B_\beta(u, v) = 1$ y $B_\beta(v, w) = 1$, entonces $B_\beta(u, w) = 1$.

2.2.3. Función objetivo

Con estas representaciones, el OBOP consiste en encontrar el *bucket order* β cuya matriz asociada B_β sea lo más cercana posible a la matriz de entrada C . Denotaremos esta distancia por D , definida como:

$$D(B_\beta, C) = \sum_{u=1}^n \sum_{v=1}^n |B_\beta(u, v) - C(u, v)|. \quad (2.2)$$

Por tanto, el problema puede expresarse como:

$$\beta^* = \arg \min_{B_\beta} D(B_\beta, C).$$

Puesto que pueden existir varias soluciones con el mismo valor objetivo, β^* representa una de las soluciones óptimas obtenidas por el procedimiento de resolución.

2.2.4. Complejidad

Desde el punto de vista computacional, el OBOP es un problema *NP-hard* [2, 4, 12]. Esta dificultad está relacionada con la propia naturaleza del espacio de búsqueda. En un ranking estricto sobre n alternativas, el número de posibles soluciones es $n!$, ya que únicamente se consideran permutaciones de las alternativas.

Sin embargo, al permitir empates, las soluciones posibles no se limitan a rankings estrictos, sino que incluyen todos los *bucket orders* definidos sobre el conjunto de alternativas. Como consecuencia, el número total de soluciones candidatas para n alternativas viene determinado por los números de Fubini [4], definidos mediante la expresión:

$$Fub(n) = \sum_{k=0}^n k! S(n, k).$$

donde $S(n, k)$ representa un número de Stirling de segunda especie, es decir, el número de formas de dividir n alternativas en k subconjuntos no vacíos. En esta expresión, $S(n, k)$ cuenta las posibles agrupaciones en *buckets*, mientras que $k!$ cuenta las posibles ordenaciones de dichos *buckets*.

Este crecimiento supera ampliamente al de $n!$, lo que hace inviable una exploración exhaustiva salvo en instancias de tamaño reducido. Esta complejidad explica la necesidad de recurrir a métodos aproximados de resolución, que serán revisados a continuación [2, 4].

2.3. Métodos de resolución del OBOP

La literatura ha abordado la resolución del OBOP desde tres enfoques principales. En primer lugar, se encuentran los algoritmos heurísticos específicos del problema, como el *Bucket Pivot Algorithm* (BPA) y sus variantes, diseñados para construir *bucket orders* de buena calidad con un coste computacional reducido [12, 13].

En segundo lugar, se han propuesto enfoques metaheurísticos, que exploran el espacio de soluciones mediante estrategias de búsqueda más generales, como las estrategias evolutivas adaptadas al OBOP [4]. Estos métodos permiten mejorar la calidad de las soluciones heurísticas, aunque normalmente a costa de un mayor tiempo de ejecución.

Por último, el enfoque exacto mediante Programación Lineal Entera permite obtener un *bucket order* óptimo respecto a la matriz de precedencias, proporcionando una garantía de optimalidad. No obstante, su aplicación queda condicionada por el tamaño de la instancia y el coste computacional asociado [5].

2.3.1. Algoritmos heurísticos y metaheurísticos

Dentro del primer enfoque, uno de los métodos más representativos para abordar el OBOP es el *Bucket Pivot Algorithm* (BPA), una heurística voraz inspirada en el algoritmo *Quicksort*. Dado un conjunto de alternativas I y una matriz de precedencias C , BPA selecciona una alternativa $p \in I$ como pivote y utiliza sus relaciones con el resto de alternativas para dividir el conjunto en tres partes: las alternativas que deben situarse antes del pivote, las que se consideran empatadas con él y las que deben situarse después. Esta clasificación se realiza mediante un umbral, que permite distinguir entre preferencias claras y situaciones cercanas al empate. Una vez obtenida la partición, el procedimiento se aplica de forma recursiva sobre los subconjuntos situados antes y después del *bucket* central, hasta construir un *bucket order* completamente definido [4, 12].

La principal ventaja del BPA reside en su sencillez y bajo coste computacional, lo que lo convierte en una opción adecuada para obtener soluciones aproximadas de forma rápida. Sin embargo, su comportamiento depende en gran medida de la elección del pivote, ya que cada partición se construye tomando como referencia una única alternativa. Esta dependencia puede provocar variabilidad entre ejecuciones y afectar a la calidad del *bucket order* obtenido, especialmente en matrices donde existen relaciones de preferencia poco claras o zonas cercanas al empate [4, 12].

Con el fin de reducir estas limitaciones, se han propuesto variantes que modifican tanto la selección del pivote como el modo en que se clasifican las alternativas durante la partición. Una de estas mejoras es la estrategia *Least Indecision Assumption* (LIA), que sustituye la selección aleatoria del pivote por un criterio más informado, dando prioridad a aquellas alternativas cuya posición relativa presenta menor incertidumbre en la matriz de precedencias. Para ello se utilizan medidas como el índice DANI, que permite estimar el grado de indecisión asociado a cada alternativa [12].

Otra línea de mejora está formada por las *estrategias multipivote*, como MP y MP2. En lugar de basar todas las decisiones en la relación de cada alternativa con un único pivote, estas variantes utilizan la información proporcionada por varias alternativas del *bucket* central. De esta forma, se reduce la dependencia de comparaciones locales y se

obtienen particiones más estables [4, 12].

Además de las heurísticas voraces, el OBOP también se ha abordado mediante técnicas metaheurísticas, como estrategias evolutivas. Estos métodos exploran de forma más amplia el espacio de posibles *bucket orders* e incorporan operadores capaces de modificar tanto la posición relativa de las alternativas como la estructura interna de los *buckets*. Aunque pueden mejorar la calidad de las soluciones obtenidas por heurísticas más simples, normalmente requieren un mayor tiempo de ejecución; es decir, permiten obtener mejores soluciones a cambio de un mayor coste computacional [4].

En conjunto, los algoritmos heurísticos y metaheurísticos permiten abordar instancias de mayor tamaño cuando el coste computacional es un factor determinante. Sin embargo, al no garantizar optimalidad, pueden dificultar la comparación entre el *bucket order* original y el *bucket order* con ruido, ya que parte de las diferencias observadas podría deberse al propio algoritmo de resolución y no únicamente al ruido introducido.

2.3.2. Formulación de Programación Lineal Entera del OBOP

Aledo et al. proponen una formulación de Programación Lineal Entera (*Integer Linear Programming*, ILP) para el OBOP, que permite obtener de forma exacta un *bucket order* óptimo respecto a la matriz de precedencias C [5]. Esta formulación se utilizará en este trabajo como base para calcular el *bucket order* óptimo de referencia.

El modelo representa las relaciones entre pares de alternativas mediante variables binarias x_{rs} , definidas para todo par de alternativas distintas $r, s \in \{1, \dots, n\}$:

$$x_{rs} = \begin{cases} 1, & \text{si } r \text{ precede o está empatado con } s, \\ 0, & \text{en caso contrario.} \end{cases}$$

Bajo esta codificación, si $x_{rs} = 1$ y $x_{sr} = 1$, las alternativas r y s pertenecen al mismo *bucket*. En cambio, si $x_{rs} = 1$ y $x_{sr} = 0$, entonces r precede estrictamente a s .

Para que estas variables definan un ranking con empates válido, se imponen restricciones de comparabilidad y transitividad. La restricción de comparabilidad viene dada por:

$$x_{rs} + x_{sr} \geq 1, \quad \forall r < s. \quad (2.3)$$

Esta restricción garantiza que cualquier par de alternativas quede relacionado: siempre una precede a la otra o ambas aparecen empatadas. Por otro lado, la transitividad se impone mediante:

$$x_{rs} + x_{st} \leq 1 + x_{rt}, \quad \forall r, s, t \text{ distintos.} \quad (2.4)$$

Esta restricción asegura la coherencia del orden. Es decir, si r precede o empata con s , y s precede o empata con t , entonces r también debe preceder o empatar con t . Además, las variables deben ser binarias:

$$x_{rs} \in \{0, 1\}, \quad \forall r \neq s. \quad (2.5)$$

Una vez definida la estructura del ranking, se introduce la matriz $B = (b_{rs})_{n \times n}$ asociada al *bucket order* candidato. Cada entrada de esta matriz representa la relación entre dos alternativas en la solución:

$$b_{rs} = \begin{cases} 1, & \text{si } r \text{ precede estrictamente a } s, \\ 0.5, & \text{si } r \text{ y } s \text{ están empatados,} \\ 0, & \text{si } s \text{ precede estrictamente a } r. \end{cases}$$

Esta matriz puede expresarse directamente a partir de las variables x_{rs} como:

$$b_{rs} = \frac{x_{rs} - x_{sr} + 1}{2} \quad (2.6)$$

El objetivo del modelo parte de la distancia definida en (2.2). Sustituyendo b_{rs} por su expresión en función de x_{rs} , y considerando únicamente los pares $r < s$ por simetría, la función objetivo puede escribirse como:

$$f(\text{OBOP}, x) = 2 \sum_{r=1}^n \sum_{s=r+1}^n \left| \frac{x_{rs} - x_{sr} + 1}{2} - c_{rs} \right|. \quad (2.7)$$

Para obtener una formulación lineal, se introducen variables auxiliares $d_{rs} \geq 0$, con $r < s$, que representan la desviación absoluta entre la relación inducida por el consenso y la información recogida en C . Así, el modelo minimiza:

$$\min 2 \sum_{r=1}^n \sum_{s=r+1}^n d_{rs} \quad (2.8)$$

sujeto a:

$$d_{rs} \geq \frac{x_{rs} - x_{sr} + 1}{2} - c_{rs}, \quad \forall r < s, \quad (2.9)$$

$$d_{rs} \geq c_{rs} - \frac{x_{rs} - x_{sr} + 1}{2}, \quad \forall r < s. \quad (2.10)$$

Estas restricciones linealizan el valor absoluto de la función objetivo. Por tanto, junto con las restricciones de comparabilidad, transitividad y binariedad, el modelo obtiene el *bucket order* óptimo respecto a la matriz de precedencias original.

En relación con las heurísticas, ambos enfoques resultan complementarios: las heurísticas priorizan la eficiencia computacional y permiten tratar instancias de mayor tamaño, mientras que la formulación exacta prioriza la garantía de optimalidad en instancias pequeñas o moderadas. En el desarrollo experimental de este trabajo se empleará la formulación exacta, con el fin de disponer de un *bucket order* óptimo de referencia para evaluar el efecto del ruido.

2.4. Privacidad en agregación de rankings

Los rankings pueden contener información sensible, ya que reflejan preferencias, valoraciones o prioridades de quienes los generan. En contextos como la recomendación, la votación o la toma de decisiones, publicar o centralizar estas preferencias puede revelar información personal, profesional o estratégica. Por ello, en la agregación de rankings no solo interesa obtener un consenso de calidad, sino también limitar la exposición de los datos originales.

La anonimización no siempre es suficiente para proteger este tipo de información. Aunque se eliminen identificadores explícitos, la combinación concreta de preferencias de un individuo puede actuar como una huella distintiva. Un ejemplo representativo es el caso de los Premios Hugo de 2015, donde se observó que datos de nominaciones anonimizados podían facilitar la reidentificación de votantes mediante información auxiliar [10].

Para abordar este problema, distintos trabajos han aplicado *privacidad diferencial* a la agregación de rankings, proponiendo mecanismos basados en reglas de agregación como Borda, algoritmos inspirados en *Quicksort* y métodos basados en el mecanismo exponencial [10, 14]. Además, también se han estudiado modelos locales y distribuidos, en los que los participantes perturban sus datos antes de enviarlos al agregador [7]. Este enfoque busca que el resultado publicado no dependa de forma significativa de la información aportada por los individuos, reduciendo así la posibilidad de inferir preferencias individuales a partir del resultado final.

Muchos de estos enfoques se han planteado para rankings estrictos, donde cada

alternativa ocupa una posición distinta. Sin embargo, en muchos problemas reales aparecen empates o niveles equivalentes de preferencia, dando lugar a rankings con empates o *bucket orders*. En estos casos, la introducción de ruido puede afectar tanto al orden relativo entre alternativas como a la formación de los propios *buckets*, lo que convierte este escenario en un caso especialmente relevante dentro del estudio de métodos de agregación privados.

En este trabajo, la privacidad se aborda desde una perspectiva experimental. La idea general consiste en introducir ruido sobre los datos utilizados en la agregación, de forma que estos no coincidan exactamente con las preferencias originales. Con ello se busca dificultar que pueda reconstruirse de forma directa el ranking aportado por cada individuo. No obstante, estas modificaciones también pueden cambiar las relaciones de precedencia entre pares de alternativas y, por tanto, afectar al *bucket order* obtenido. Por ello, el objetivo es analizar el equilibrio entre el nivel de ruido aplicado y la conservación de la calidad del *bucket order* final.

Para estudiar este efecto, se consideran tres escenarios principales de introducción de ruido. El primero actúa directamente sobre la matriz de precedencias C . El segundo modifica los rankings individuales antes de construir la matriz C agregada. El tercero introduce ruido sobre una representación intermedia basada en *scores* latentes, a partir de los cuales se reconstruye una nueva matriz C . Además, en el siguiente apartado se introduce el Aprendizaje Federado como escenario complementario, donde la introducción de ruido se aplica de forma local antes de la agregación global.

En este capítulo se presenta únicamente la idea general de estas aproximaciones. La definición concreta de cada estrategia, su implementación y las métricas utilizadas para evaluarlas se desarrollan con más detalle en el capítulo siguiente.

2.5. Aprendizaje Federado

El *Aprendizaje Federado* (*Federated Learning*, AF) es un paradigma de computación distribuida en el que varios clientes colaboran para obtener un modelo o resultado global sin centralizar directamente sus datos locales. A diferencia del enfoque centralizado, donde todos los datos se recopilan en un único servidor, en un escenario federado cada cliente mantiene sus datos en local y solo comunica al servidor actualizaciones, parámetros o representaciones derivadas de ellos [15, 16].

La idea general puede entenderse como un proceso iterativo. El servidor define una tarea común y envía a los clientes la información necesaria para realizar el cálculo local.

Cada cliente utiliza sus propios datos para obtener una actualización local y la envía al servidor, que combina las actualizaciones recibidas para construir una nueva solución global. Este procedimiento puede repetirse durante varias rondas hasta alcanzar un resultado suficientemente estable [15]. La Figura 2.1 resume este esquema general.

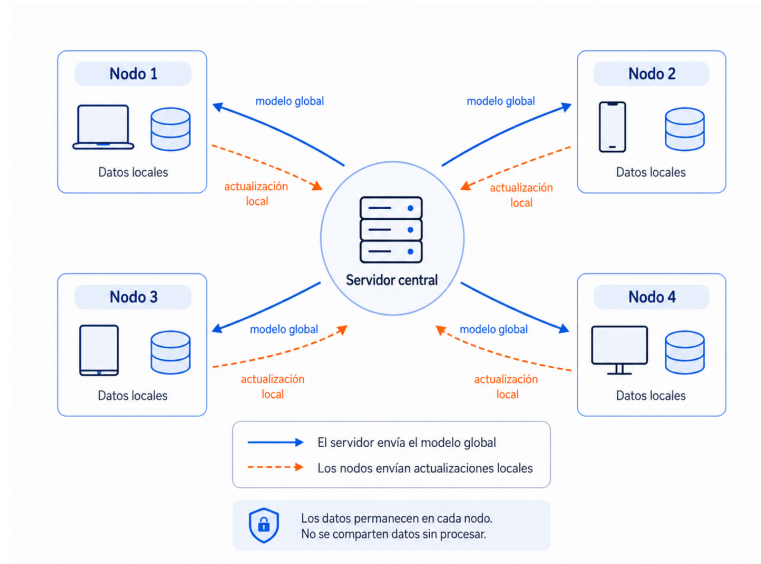


Figura 2.1: Esquema general del aprendizaje federado. Los datos permanecen almacenados localmente en los clientes, mientras que el servidor central recibe y agrega únicamente las actualizaciones de los modelos locales. Imagen generada mediante inteligencia artificial.

Desde el punto de vista de la privacidad, el interés del Aprendizaje Federado reside en que reduce la necesidad de reunir todos los datos originales en un único punto. Esto resulta especialmente relevante cuando la información procede de usuarios, dispositivos o instituciones distintas. No obstante, el hecho de no centralizar los datos no garantiza por sí solo una protección completa: las actualizaciones enviadas al servidor pueden revelar información sobre los datos locales, por lo que el enfoque federado suele combinarse con mecanismos adicionales de protección, como perturbación local, agregación segura o privacidad diferencial [16].

Este planteamiento se adapta de forma natural a la agregación de rankings. En el enfoque centralizado, todos los rankings individuales se recopilan en un servidor, se construye una matriz de precedencias global C y se resuelve el problema de agregación sobre dicha matriz. En cambio, en un escenario federado, los rankings permanecen distribuidos entre varios clientes. Cada cliente trabaja con sus propios rankings y construye

una representación local de sus preferencias. El servidor combina esas representaciones para obtener una matriz C global sobre la que realizar la agregación.

En la literatura reciente se ha estudiado la *agregación federada de rankings* (*Federated Rank Aggregation*, FRA), cuyo objetivo es combinar rankings almacenados de forma distribuida sin acceder directamente a los rankings individuales de cada cliente. Algunos trabajos han propuesto protocolos federados basados en reglas como Borda o en codificaciones de Lehmer, especialmente bajo modelos probabilísticos de Mallows [6]. Aunque estos enfoques se centran principalmente en rankings estrictos y no en *bucket orders*, son relevantes porque muestran cómo trasladar la agregación de preferencias ordinales a un escenario distribuido.

La relación entre agregación federada y privacidad también aparece en los enfoques de privacidad diferencial local o distribuida aplicados a rankings. En estos modelos, los participantes modifican sus preferencias antes de enviarlas al agregador, de modo que este no recibe directamente los rankings originales [10]. Esta idea conecta con el objetivo de este trabajo: estudiar cómo afecta la introducción de ruido cuando la agregación no se realiza únicamente sobre una matriz global centralizada, sino también sobre representaciones locales calculadas por distintos clientes.

En el contexto concreto del OBOP, el enfoque federado puede formularse a partir de matrices C locales. Los rankings se distribuyen entre K clientes, cada uno de los cuales construye y modifica su matriz C_i antes de enviarla al servidor. Este combina las matrices recibidas para obtener $\tilde{C}_{fed}$ y resolver el OBOP. A diferencia del aprendizaje federado convencional, donde el entrenamiento suele desarrollarse durante varias rondas, esta propuesta no entrena un modelo ni ejecuta un procedimiento iterativo, sino que realiza una única ronda de intercambio entre los clientes y el servidor. Aún así, conserva el principio fundamental de mantener los datos originales en los clientes y compartir únicamente representaciones derivadas y perturbadas. Este flujo se representa en la Figura 2.2.

Este esquema permite comparar dos formas de introducir ruido. En el enfoque centralizado, el ruido se aplica sobre rankings o matrices construidas a partir de todos los datos. En el enfoque federado, en cambio, el ruido se introduce localmente antes de la agregación global. Por tanto, no solo interesa estudiar cuánto ruido se introduce, sino también en qué punto del proceso se aplica y cómo afecta esto a la calidad del *bucket order* final. La implementación concreta de esta simulación federada y las métricas utilizadas para compararla con el enfoque centralizado se presentan en el capítulo siguiente.

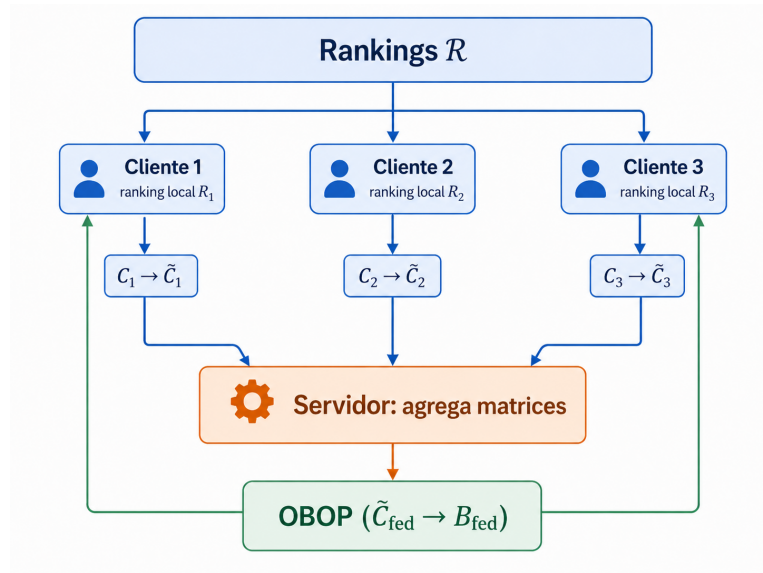


Figura 2.2: Esquema de agregación federada aplicado al OBOP con introducción de ruido local. Cada cliente construye y modifica su matriz local antes de la agregación global. Imagen generada mediante inteligencia artificial.

3. Metodología experimental

Una vez introducidas las bases teóricas del problema, en este capítulo se presenta la metodología seguida para el desarrollo experimental del trabajo. El objetivo principal es estudiar el efecto de distintas *estrategias de introducción de ruido* en problemas de agregación de rankings con empates, analizando en qué medida permiten proteger parcialmente la información inicial sin alterar de forma significativa el *bucket order* obtenido.

Para ello, se parte de una base de datos compuesta por distintas instancias de rankings definidos sobre un conjunto de ítems $I = \{1, \dots, n\}$, donde n representa el número de alternativas. A partir de dichos rankings se construye una matriz de precedencias C , que resume la información de preferencia entre cada par de alternativas.

Sobre esta matriz original se resuelve el OBOP mediante Programación Lineal Entera (ILP), obteniendo un *bucket order* óptimo de referencia β^* . A continuación, se aplica una estrategia de introducción de ruido con el objetivo de incorporar una capa adicional de privacidad sobre las preferencias originales. Esta perturbación hace que los rankings iniciales, o la información derivada directamente de ellos, no se utilicen sin modificar en la resolución posterior, reduciendo así la exposición directa de las preferencias individuales y dificultando su posible reconstrucción o manipulación.

La comparación entre β^* y $\tilde{\beta}$ permite evaluar el impacto de cada estrategia de perturbación. Dado que en cada configuración experimental se resuelve el OBOP mediante ILP, tanto sobre la matriz original C como sobre la matriz perturbada $\tilde{C}$, la viabilidad computacional depende principalmente del número de alternativas de la instancia. Por este motivo, los experimentos se plantean sobre instancias con un número de alternativas reducido o moderado, manteniendo así tiempos de resolución asumibles y permitiendo repetir las ejecuciones para distintos niveles de ruido, técnicas y semillas aleatorias.

En las siguientes secciones se describen las estrategias de introducción de ruido implementadas, las métricas utilizadas para evaluar los resultados y el procedimiento general seguido para llevar a cabo los experimentos.

Denotaremos por β^* el *bucket order* óptimo obtenido sobre C y por $B^* = B_{\beta^*}$ su matriz asociada. De forma análoga, $\tilde{\beta}$ representa el *bucket order* obtenido sobre $\tilde{C}$, mientras que $\tilde{B} = B_{\tilde{\beta}}$ representa su matriz asociada.

3.1. Estrategias de introducción de ruido

En este Trabajo de Fin de Grado se han implementado tres familias de estrategias de introducción de ruido, donde cada una de ellas actúa sobre una representación distinta de la información de preferencias. La primera introduce ruido directamente sobre la matriz de precedencias C . La segunda modifica los rankings individuales antes de construir dicha matriz. La tercera utiliza una representación intermedia basada en *scores* latentes, sobre los que se introduce ruido antes de reconstruir una nueva matriz de precedencias.

Estas estrategias se plantean como mecanismos experimentales para modificar de forma controlada la información utilizada por el OBOP. Aunque están motivadas por la idea de reducir la exposición directa de las preferencias originales, en este trabajo no se proporciona una garantía formal de ε -privacidad diferencial. Por ello, el parámetro b se utiliza como parámetro experimental para controlar la intensidad de la introducción de ruido, y su efecto real se evalúa posteriormente mediante la distancia de ruido definida en la Sección 3.4.

En todos los casos, el objetivo final es obtener una matriz perturbada $\tilde{C}$ sobre la que pueda resolverse de nuevo el OBOP y comparar el *bucket order* resultante con el *bucket order* inicial.

3.1.1. Ruido directo sobre la matriz

La primera estrategia consiste en introducir ruido directamente sobre la matriz de precedencias $C(u, v)$, que es la representación agregada utilizada como entrada del OBOP. En este caso, la perturbación se aplica sobre las comparaciones por pares de la matriz. En particular, para cada par de alternativas $u < v$ ¹, se calcula:

¹Como la matriz de precedencias cumple la propiedad de complementariedad $C(u, v) + C(v, u) = 1$, no es necesario perturbar por separado las dos entradas asociadas a un mismo par de alternativas.

$$\tilde{C}(u, v) = C(u, v) + \eta_{uv}, \quad (3.1)$$

donde η_{uv} es un valor de ruido generado aleatoriamente a partir de una distribución de Laplace centrada en cero. En la implementación, este valor se obtiene mediante el generador aleatorio de NumPy [17], usando una instrucción equivalente a:

$$\eta_{uv} \sim \text{Laplace}(0, b).$$

Por tanto, η_{uv} no se calcula de forma determinista a partir de b , sino que se obtiene como una muestra aleatoria de una distribución cuya escala viene dada por dicho parámetro. Además, la semilla aleatoria utilizada en cada ejecución permite reproducir los valores generados.

La distribución de Laplace con parámetro de escala $b > 0$ tiene función de densidad dada por:

$$f(x) = \frac{1}{2b} \exp\left(-\frac{|x|}{b}\right). \quad (3.2)$$

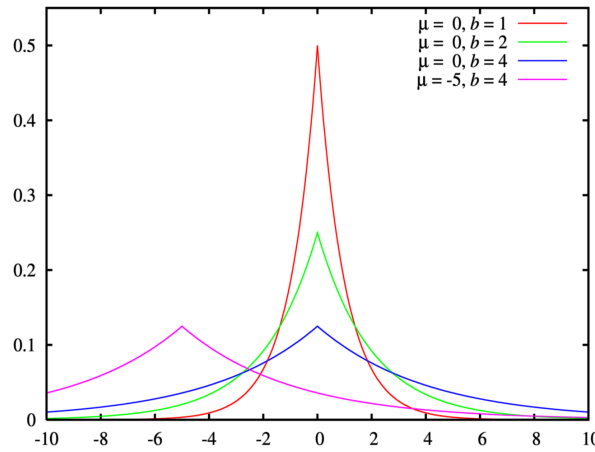


Figura 3.1: Función de densidad de la distribución de Laplace. Fuente: [18], bajo licencia CC BY-SA 3.0.

Como se observa en la Figura 3.1, la distribución de Laplace es simétrica respecto a cero. Por ello, el ruido generado puede ser positivo o negativo, de modo que una comparación $C(u, v)$ puede aumentar o disminuir tras la perturbación. Además, la distribución concentra mayor probabilidad en valores cercanos a cero, por lo que las

perturbaciones pequeñas son más frecuentes que las perturbaciones grandes.

En este trabajo, el parámetro b se interpreta como intensidad de perturbación. Cuando b toma valores pequeños, la distribución está más concentrada alrededor de cero y, por tanto, los valores de η_{uv} suelen ser reducidos. En cambio, al aumentar b , la distribución se vuelve más dispersa y aumenta la probabilidad de obtener perturbaciones de mayor magnitud.

Aunque el uso de ruido de Laplace está inspirado en mecanismos empleados en privacidad diferencial [14, 19], en este trabajo no se proporciona una garantía formal de ε -privacidad diferencial.

Ejemplo: supongamos que $C(1, 3) = 0.52$, lo que indica una preferencia ligera de la alternativa 1 sobre la alternativa 3. Si en una ejecución concreta se genera $\eta_{13} = -0.07$, entonces $\tilde{C}(1, 3) = 0.45$ y, por complementariedad, $\tilde{C}(3, 1) = 0.55$.

Una vez añadido el ruido, cada valor $C(u, v)$ se proyecta sobre el intervalo $[0, 1]$ mediante: $\tilde{C}(u, v) = \max(0, \min(1, C(u, v) + \eta_{uv}))$. Después, se define $\tilde{C}(v, u) = 1 - \tilde{C}(u, v)$, y se fija $\tilde{C}(u, u) = 0.5$ para todo $u \in I$. De este modo, la matriz perturbada $\tilde{C}$ cumple las propiedades de una matriz de precedencias:

$$0 \leq \tilde{C}(u, v) \leq 1, \quad \tilde{C}(u, v) + \tilde{C}(v, u) = 1. \quad (3.3)$$

Ejemplo: si $C(u, v) = 0.90$ y el ruido generado es $\eta_{uv} = 0.18$, inicialmente se obtiene $C(u, v) + \eta_{uv} = 1.08$. Como las entradas de la matriz deben pertenecer al intervalo $[0, 1]$, el valor se recorta a $\tilde{C}(u, v) = 1$, y por tanto, $\tilde{C}(v, u) = 0$.

Dentro de esta estrategia se han considerado tres técnicas de perturbación. En los tres casos, cuando una comparación es seleccionada, el ruido añadido se genera del mismo modo, es decir, mediante una muestra de una distribución de Laplace centrada en cero y con parámetro de escala b . Por tanto, la diferencia entre las técnicas no está en el tipo de ruido utilizado, sino en el conjunto de pares de alternativas sobre los que se aplica.

- **Ruido en toda la matriz (todos):** se introduce ruido en todas las comparaciones por pares de la matriz. Para cada par (u, v) , con $u < v$, se genera un valor de ruido independiente η_{uv} y se modifica la entrada $C(u, v)$.

Ejemplo. Si una matriz contiene cuatro alternativas $I = \{1, 2, 3, 4\}$, existen seis comparaciones independientes: $(1, 2)$, $(1, 3)$, $(1, 4)$, $(2, 3)$, $(2, 4)$ y $(3, 4)$. Con esta técnica se modificarían todas esas comparaciones.

- **Ruido aleatorio (aleatoria):** se selecciona aleatoriamente un subconjunto de

pares y solo se introduce ruido en dichas entradas. En la implementación se fija una probabilidad de selección:

$$p_{\text{aleatoria}} = 0.30.$$

Esto significa que cada par de alternativas tiene un 30% de probabilidad de ser perturbado y un 70% de probabilidad de mantenerse sin cambios. Este parámetro se mantiene fijo durante los experimentos para evitar añadir una dimensión adicional al estudio. De este modo, el análisis se centra principalmente en el efecto del parámetro b , que controla la intensidad del ruido aplicado a los pares que sí han sido seleccionados.

Ejemplo. Si una matriz contiene cuatro alternativas $I = \{1, 2, 3, 4\}$, existen seis comparaciones independientes: $(1, 2)$, $(1, 3)$, $(1, 4)$, $(2, 3)$, $(2, 4)$ y $(3, 4)$. Con esta técnica, una ejecución concreta podría seleccionar únicamente los pares $(1, 3)$ y $(2, 4)$, dejando el resto sin modificar. Sin embargo, con otra semilla aleatoria podrían seleccionarse otros pares distintos.

- **Ruido cerca del empate** (`cerca_empate`): se introduce ruido únicamente en aquellas comparaciones cuyo valor está próximo a 0.5. Estas entradas representan situaciones en las que no existe una preferencia clara entre dos alternativas. En la implementación se fija el umbral:

$$\theta_{\text{empate}} = 0.10.$$

Por tanto, un par (u, v) se selecciona para ser modificado si cumple:

$$|C(u, v) - 0.5| \leq 0.10.$$

De forma equivalente, se modifican las comparaciones cuyo valor pertenece al intervalo:

$$C(u, v) \in [0.4, 0.6].$$

Ejemplo. Una comparación con $C(u, v) = 0.52$ sí se modifica, porque está cerca del empate y pertenece al intervalo anterior. En cambio, una comparación con $C(u, v) = 0.85$ no se modifica, porque representa una preferencia mucho más clara de u sobre v .

Esta técnica parte de la hipótesis de que las comparaciones próximas a 0.5 contienen una mayor incertidumbre.

Por tanto, la técnica utilizada determina *dónde* se introduce ruido. El parámetro b controla la escala de los valores de ruido generados, mientras que la semilla fija la realización aleatoria concreta, es decir, los valores η_{uv} añadidos en cada comparación modificada. En la técnica **aleatoria**, además, la semilla también influye en qué pares son seleccionados para ser modificados.

El Algoritmo 3.1 resume el procedimiento general seguido para introducir ruido directo sobre la matriz de precedencias.

Algoritmo 3.1: Pseudo-código de la estrategia de introducción de ruido sobre la matriz $C(u, v)$.

Entrada: Matriz de precedencias C , conjunto de alternativas I , nivel de ruido b , técnica de perturbación, semilla aleatoria

Salida: Matriz de precedencias $\tilde{C}$

- 1: Inicializar el generador aleatorio con la semilla indicada
 - 2: Inicializar $\tilde{C} \leftarrow C$
 - 3: **para** cada par (u, v) , con $u < v$ **hacer**
 - 4: **si** el par (u, v) cumple la técnica de perturbación **entonces**
 - 5: Generar $\eta_{uv} \sim \text{Laplace}(0, b)$
 - 6: $\tilde{C}(u, v) \leftarrow C(u, v) + \eta_{uv}$
 - 7: Recortar $\tilde{C}(u, v)$ al intervalo $[0, 1]$
 - 8: $\tilde{C}(v, u) \leftarrow 1 - \tilde{C}(u, v)$
 - 9: **si no**
 - 10: $\tilde{C}(u, v) \leftarrow C(u, v)$
 - 11: $\tilde{C}(v, u) \leftarrow C(v, u)$
 - 12: **para** cada alternativa $u \in I$ **hacer**
 - 13: $\tilde{C}(u, u) \leftarrow 0.5$
 - 14: **devolver** $\tilde{C}$
-

3.1.2. Ruido directo sobre rankings individuales

La segunda estrategia consiste en introducir ruido en los rankings individuales antes de construir la matriz de precedencias. A diferencia del ruido directo sobre la matriz, en este caso no se modifica la matriz agregada C , sino la colección de rankings de entrada.

Una vez alterados, se construye a partir de ellos una nueva matriz de precedencias $\tilde{C}$, que se utilizará posteriormente como entrada del OBOP.

Esta estrategia requiere disponer de los rankings individuales originales. Aunque la entrada habitual del OBOP es la matriz de precedencias C , en este caso se trabaja en una fase anterior a su construcción. Por tanto, si únicamente se dispusiera de C , esta estrategia no podría aplicarse. Aunque se parte de distintos formatos de rankings, al introducir ruido se asume que la representación interna de cada ranking es una lista de *buckets*, donde cada *bucket* contiene un conjunto de alternativas empatadas. De este modo, se pueden aplicar operaciones elementales sobre la estructura ordinal de cada ranking, como intercambiar alternativas, mover una alternativa a otro *bucket* o modificar los empates.

Sea $R = \{R_1, \dots, R_m\}$ la colección original de rankings, donde m representa el número de rankings disponibles. El objetivo es obtener una colección perturbada $\tilde{R} = \{\tilde{R}_1, \dots, \tilde{R}_m\}$, a partir de la cual se construye la matriz $\tilde{C}$ siguiendo el mismo procedimiento utilizado para construir la matriz original, descrito en el Anexo A.

En esta estrategia no se añade ruido aditivo de Laplace a las entradas de la matriz. En su lugar, se aplican operaciones discretas sobre la estructura de los rankings. Por ello, el parámetro $b \in [0, 1]$ no representa la escala de una distribución de ruido, sino la probabilidad de seleccionar cada ranking para ser perturbado.

En concreto, para cada ranking R_i se genera un número aleatorio $s_i \in [0, 1]$. Si $s_i < b$, el ranking se modifica; en caso contrario, se conserva sin cambios. Así, por ejemplo, si $b = 0.30$, cada ranking tiene un 30% de probabilidad de ser perturbado.

Una vez seleccionado un ranking, se aplica un número de cambios proporcional a su tamaño. En la implementación se fija una proporción del 10% de las alternativas del ranking. Por tanto, si un ranking contiene n_i alternativas, el número de cambios aplicados se calcula como:

$$q_i = \begin{cases} 0, & \text{si } n_i < 2, \\ \max(1, \lceil 0.10 \cdot n_i \rceil), & \text{si } n_i \geq 2. \end{cases}$$

De este modo, aumentar b no implica que cada ranking seleccionado sufra una modificación más intensa, sino que aumenta la proporción esperada de rankings modificados dentro de la colección. La intensidad local de la modificación viene determinada por el número de cambios q_i aplicados a cada ranking seleccionado.

Cuando un ranking es seleccionado para ser perturbado, se elige aleatoriamente una operación elemental entre cuatro posibilidades:

-
- **Intercambio:** se seleccionan aleatoriamente dos *buckets* distintos y se escoge una alternativa de cada uno. A continuación, ambas alternativas se intercambian, de forma que cada una pasa al *bucket* donde estaba la otra.

Ejemplo: si se parte del ranking

$$R_i = 1 \mid \{2, 3\} \mid \{4, 5\}$$

y se seleccionan las alternativas 1 y 4, el nuevo ranking queda como:

$$\tilde{R}_i = 4 \mid \{2, 3\} \mid \{1, 5\}.$$

- **Movimiento:** se selecciona aleatoriamente una alternativa y se desplaza a otro *bucket*. Para ello, se elige de forma aleatoria un desplazamiento entre 1 y el número de *buckets* menos uno, así como una dirección de movimiento. La nueva posición se calcula de forma que siempre corresponda a un *bucket* válido.

Ejemplo: Si se parte de

$$R_i = 1 \mid \{2, 3\} \mid \{4, 5\}$$

y se selecciona la alternativa 4 con desplazamiento 1 hacia el *bucket* anterior, se obtiene:

$$\tilde{R}_i = 1 \mid \{2, 3, 4\} \mid 5.$$

- **Modificación de empates:** se selecciona aleatoriamente un *bucket* del ranking. Si contiene varias alternativas, se elige aleatoriamente una de ellas, se extrae del *bucket* y se inserta como un nuevo *bucket* en una posición también aleatoria, rompiendo parcialmente el empate. Si el *bucket* seleccionado contiene una única alternativa, se elige otro *bucket*, se extrae aleatoriamente una alternativa de él y se añade al *bucket* seleccionado, creando o ampliando un empate.

Ejemplo: si se parte de

$$R_i = 1 \mid \{2, 3\} \mid \{4, 5\}$$

y se selecciona el *bucket* $\{2, 3\}$, puede elegirse la alternativa 2 y separarse en un nuevo *bucket*:

$$\tilde{R}_i = 1 \mid 2 \mid 3 \mid \{4, 5\}.$$

En este caso, se rompe el empate entre 2 y 3.

- **Perturbación local:** se selecciona aleatoriamente un tramo consecutivo de *buc-*
-

kets y se invierte su orden. El tramo contiene al menos dos *buckets*, por lo que la operación modifica el orden relativo entre ellos, pero conserva los empates internos dentro de cada *bucket*.

Ejemplo: si se parte de

$$R_i = 1 \mid \{2, 3\} \mid \{4, 5\}$$

y se invierte el tramo formado por los dos primeros *buckets*, se obtiene:

$$\tilde{R}_i = \{2, 3\} \mid 1 \mid \{4, 5\}.$$

En todos los casos, la semilla controla la selección aleatoria de los rankings a modificar, la operación elemental aplicada y las alternativas o *buckets* afectados. Si un ranking está formado por un único *bucket*, solo se permite la operación de modificación de empates, ya que las operaciones de intercambio, movimiento y perturbación local no alterarían la estructura de precedencias.

El Algoritmo 3.2 resume el procedimiento general seguido para introducir ruido sobre los rankings individuales.

Algoritmo 3.2: Pseudo-código de la estrategia de introducción de ruido sobre los rankings individuales.

Entrada: Colección de rankings $R = \{R_1, \dots, R_m\}$, conjunto de alternativas I , probabilidad de perturbación b , semilla aleatoria

Salida: Matriz de precedencias $\tilde{C}$

```

1: Inicializar el generador aleatorio con la semilla indicada
2: Inicializar  $\tilde{R} \leftarrow \emptyset$ 
3: para cada ranking  $R_i \in R$  hacer
4:     Generar  $s_i$  aleatoriamente en el intervalo  $[0, 1]$ 
5:     si  $s_i < b$  entonces
6:         Obtener  $n_i$ , el número de alternativas de  $R_i$ 
7:         si  $n_i < 2$  entonces
8:              $\tilde{R}_i \leftarrow R_i$ 
9:         si no
10:            Calcular  $q_i \leftarrow \max(1, \lceil 0.10 \cdot n_i \rceil)$ 
11:            si  $R_i$  contiene un único bucket entonces
12:                Seleccionar la operación modificación de empates
13:            si no
14:                Seleccionar aleatoriamente una operación elemental entre intercambio, movimiento, empate y perturbación local
15:            Aplicar la operación seleccionada  $q_i$  veces sobre  $R_i$  para obtener  $\tilde{R}_i$ 
16:            Eliminar buckets vacíos de  $\tilde{R}_i$ 
17:        si no
18:             $\tilde{R}_i \leftarrow R_i$ 
19:        Añadir  $\tilde{R}_i$  a  $\tilde{R}$ 
20: Construir  $\tilde{C}$  a partir de la colección perturbada  $\tilde{R}$ 
21: devolver  $\tilde{C}$ 

```

3.1.3. Ruido sobre scores latentes y reconstrucción de la matriz

La tercera estrategia parte de la idea de que la matriz $C(u, v)$ puede aproximarse a partir de ciertos *scores* latentes $s(u)$ para cada alternativa $u \in I$. Es decir, en lugar de introducir ruido directamente en las entradas de la matriz C , se resume primero la información de cada alternativa en una única puntuación.

Para cada alternativa $u \in I$, el *score* se calcula como la suma de la fila correspondiente en la matriz de precedencias:

$$s(u) = \sum_{v \in I} C(u, v). \quad (3.4)$$

Este valor resume, de forma agregada, cuánto tiende la alternativa u a preceder al resto de alternativas. Por tanto, un *score* alto indica que u aparece como preferida frente a muchas alternativas, mientras que un *score* bajo indica lo contrario.

Para evitar que su escala dependa del tamaño de la instancia, se aplica una estandarización tipo *z-score*. Para ello, a cada *score* se le resta la media y se divide por la desviación típica:

$$z(u) = \frac{s(u) - \bar{s}}{s_{\text{std}}}, \quad (3.5)$$

donde $\bar{s}$ es la media de los *scores* y s_{std} su desviación típica. Si $s_{\text{std}} = 0$, todas las alternativas tienen el mismo *score*. En este caso, se asigna $z(u) = 0$ para todo $u \in I$. Por tanto, cualquier diferencia posterior entre los *scores* procederá exclusivamente del ruido añadido.

Posteriormente, se añade ruido de Laplace independiente a cada *score* estandarizado:

$$\tilde{z}(u) = z(u) + \eta_u, \quad \eta_u \sim \text{Laplace}(0, b). \quad (3.6)$$

En esta estrategia, el parámetro b vuelve a actuar como intensidad de perturbación, pero ahora sobre los *scores* estandarizados. Valores pequeños de b generan *scores* perturbados más próximos a los *scores* originales, mientras que valores más altos pueden alterar con más fuerza la posición relativa de las alternativas.

Una vez introducido el ruido, se reconstruye una nueva matriz $\tilde{C}$ comparando los *scores* perturbados mediante una función de reconstrucción σ :

$$\tilde{C}(u, v) = \sigma(\tilde{z}(u) - \tilde{z}(v)). \quad (3.7)$$

La función σ transforma la diferencia entre *scores* en un valor entre 0 y 1. En este trabajo se han considerado dos tipos de funciones:

■ **Logística:**

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

■ **Probit:**

$$\sigma(x) = \Phi(x),$$

donde Φ es la función de distribución acumulada de la normal estándar.

Ambas funciones devuelven valores en el intervalo $[0, 1]$ y cumplen que $\sigma(0) = 0.5$. Por tanto, una diferencia nula entre *scores* se transforma en una comparación empatada, mientras que diferencias positivas o negativas generan preferencias a favor de una u otra alternativa.

Ejemplo: para ilustrar la reconstrucción, supongamos que después de añadir ruido se obtienen los siguientes *scores* perturbados para tres alternativas:

$$\tilde{z}(1) = 0.80, \quad \tilde{z}(2) = 0.10, \quad \tilde{z}(3) = -0.60.$$

Usando la función logística, las comparaciones independientes se calculan como:

$$\tilde{C}(1, 2) = \sigma(0.80 - 0.10) = \sigma(0.70) \approx 0.67,$$

$$\tilde{C}(1, 3) = \sigma(0.80 - (-0.60)) = \sigma(1.40) \approx 0.80,$$

$$\tilde{C}(2, 3) = \sigma(0.10 - (-0.60)) = \sigma(0.70) \approx 0.67.$$

A partir de estos valores, la matriz reconstruida queda aproximadamente:

$$\tilde{C} = \begin{pmatrix} 0.5 & 0.67 & 0.80 \\ 0.33 & 0.5 & 0.67 \\ 0.20 & 0.33 & 0.5 \end{pmatrix}.$$

Este ejemplo muestra que el ruido no se introduce entrada a entrada, sino sobre una representación intermedia. Por ello, modificar el *score* de una alternativa puede afectar simultáneamente a varias comparaciones de la matriz reconstruida.

Esta estrategia debe interpretarse con cuidado, ya que incluso cuando $b = 0$, la matriz reconstruida puede no coincidir exactamente con la matriz original C . Esto se debe a que la información de la matriz se comprime en un único *score* por alternativa antes de reconstruir $\tilde{C}$.

El Algoritmo 3.3 resume el procedimiento seguido.

Algoritmo 3.3: Pseudo-código de la estrategia de introducción de ruido sobre *scores* latentes.

Entrada: Matriz original C , conjunto de alternativas I , nivel de ruido b , función de reconstrucción σ , semilla aleatoria

Salida: Matriz de precedencias $\tilde{C}$.

- 1: Inicializar el generador aleatorio con la semilla indicada
 - 2: Calcular $s(u) = \sum_{v \in I} C(u, v)$ para cada alternativa $u \in I$
 - 3: Estandarizar los *scores* $z(u)$ para todo $u \in I$
 - 4: **para** cada alternativa $u \in I$ **hacer**
 - 5: Generar $\eta_u \sim \text{Laplace}(0, b)$
 - 6: $\tilde{z}(u) \leftarrow z(u) + \eta_u$
 - 7: **para** cada par (u, v) , con $u < v$ **hacer**
 - 8: $\tilde{C}(u, v) \leftarrow \sigma(\tilde{z}(u) - \tilde{z}(v))$
 - 9: $\tilde{C}(v, u) \leftarrow 1 - \tilde{C}(u, v)$
 - 10: **para** cada alternativa $u \in I$ **hacer**
 - 11: $\tilde{C}(u, u) \leftarrow 0.5$
 - 12: **devolver** $\tilde{C}$
-

3.2. Procedimiento centralizado con ruido

En primer lugar, para cada instancia seleccionada se cargan los rankings y se construye la matriz de precedencias original C . Sobre esta matriz se resuelve una única vez el OBOP mediante Programación Lineal Entera, obteniendo el *bucket order* original β^* y su valor objetivo $D(B^*, C)$. Este resultado constituye el *baseline* centralizado de la instancia y se reutiliza como referencia en todas las configuraciones de ruido ejecutadas sobre ella.

A continuación, se fija una configuración experimental determinada por la estrategia de introducción de ruido, la técnica concreta, el valor del parámetro b y la semilla aleatoria. La estrategia indica sobre qué representación se introduce ruido: la matriz de precedencias, la colección de rankings individuales o los *scores* latentes. La técnica especifica la variante aplicada dentro de cada estrategia, por ejemplo introducir ruido en toda la matriz, seleccionar aleatoriamente algunas comparaciones, actuar sobre comparaciones cercanas al empate o reconstruir la matriz mediante una función logística o probit.

El parámetro b se utiliza como parámetro experimental para controlar la intensidad de la introducción de ruido, aunque su interpretación concreta depende de la estrategia

utilizada. Por su parte, la semilla aleatoria permite reproducir una ejecución concreta. Dado que varias estrategias incorporan decisiones aleatorias, una misma configuración se repite con distintas semillas. De este modo, los resultados no dependen únicamente de una realización particular del ruido, sino que pueden analizarse de forma más estable.

Para cada configuración se obtiene una nueva matriz con ruido $\tilde{C}$. Sobre esta matriz, se resuelve de nuevo el OBOP para obtener el nuevo *bucket order* $\tilde{\beta}$. Finalmente, se calculan las métricas definidas posteriormente en la Sección 3.4. Los resultados de cada ejecución se almacenan para su posterior análisis en el capítulo siguiente.

El procedimiento centralizado seguido para una instancia y una configuración concreta se resume en el Algoritmo 3.4.

Algoritmo 3.4: Procedimiento experimental centralizado.

Entrada: Instancia de rankings, estrategia de ruido, técnica, nivel de ruido b , semilla aleatoria

Salida: Resultados experimentales de la configuración

- 1: Cargar los rankings de la instancia
 - 2: Construir la matriz de precedencias original C
 - 3: Resolver el OBOP sobre C
 - 4: Inicializar el generador aleatorio con la semilla indicada
 - 5: Generar $\tilde{C}$ según la estrategia, la técnica y el valor de b
 - 6: Resolver el OBOP sobre $\tilde{C}$
 - 7: Calcular métricas de evaluación y guardar los resultados de la configuración
-

3.3. Simulación federada

Además de los experimentos centralizados, se implementa una simulación federada para estudiar el comportamiento del OBOP cuando los rankings no se agregan directamente en un único nodo central. En este escenario, la colección de rankings se reparte entre varios clientes y cada uno trabaja únicamente con su subconjunto local de datos.

La simulación no representa un despliegue real cliente-servidor, sino una reproducción controlada del flujo federado dentro del entorno experimental. Su objetivo es comparar el *bucket order* federado con el obtenido en el escenario centralizado y analizar el efecto de introducir ruido local antes de la agregación.

En esta simulación, el ruido se introduce exclusivamente sobre las matrices de precedencias locales C_i . Se reutilizan las tres técnicas definidas para el ruido directo sobre matriz: aplicación sobre todos los pares observados, selección aleatoria de pares y selección de comparaciones cercanas a 0.5. Las estrategias aplicadas sobre rankings individuales y sobre *scores* agregados no forman parte del escenario federado estudiado.

Sea $R = \{R_1, \dots, R_m\}$ la colección de rankings de una instancia. Esta colección se divide aleatoriamente entre K clientes:

$$R = R^{(1)} \dot{\cup} R^{(2)} \dot{\cup} \dots \dot{\cup} R^{(K)}.$$

Cada cliente i construye dos matrices locales sobre el conjunto común de alternativas I . La matriz W_i acumula la evidencia de precedencia entre pares de alternativas, mientras que M_i registra cuántos rankings locales contienen simultáneamente cada par. Para $u \neq v$, se definen como:

$$W_i(u, v) = \sum_{\substack{R_j \in R^{(i)} \\ u, v \in R_j}} w_{R_j}(u, v), \quad M_i(u, v) = \sum_{R_j \in R^{(i)}} \mathbb{I}\{u, v \in R_j\},$$

donde $w_{R_j}(u, v) = 1$ si u precede a v , $w_{R_j}(u, v) = 0.5$ si ambas alternativas están empatadas y $w_{R_j}(u, v) = 0$ si v precede a u . A partir de estas matrices se obtiene la matriz local de precedencias:

$$C_i(u, v) = \begin{cases} \frac{W_i(u, v)}{M_i(u, v)}, & \text{si } M_i(u, v) > 0, \\ 0.5, & \text{si } M_i(u, v) = 0, \end{cases} \quad u \neq v,$$

y se fija $C_i(u, u) = 0.5$. Cuando $M_i(u, v) = 0$, el valor 0.5 no representa un empate observado, sino ausencia de información sobre ese par.

Para distinguir los pares observados de los no observados se define la máscara binaria:

$$P_i(u, v) = \begin{cases} 1, & \text{si } M_i(u, v) > 0, \\ 0, & \text{si } M_i(u, v) = 0. \end{cases}$$

La máscara P_i permite identificar los pares observados por cada cliente. Se utiliza para aplicar ruido únicamente sobre dichos pares y, de forma complementaria, para calcular las métricas locales.

Una vez construida la matriz local, cada cliente introduce ruido sobre C_i , obteniendo $\tilde{C}_i$. El ruido se aplica únicamente sobre los pares observados, es decir, aquellos con

$P_i(u, v) = 1$. Los pares sin información permanecen con valor 0.5.

Cada cliente envía al servidor la matriz perturbada $\tilde{C}_i$ y la matriz M_i . Con las contribuciones recibidas, el servidor construye la matriz federada mediante una agregación ponderada por par:

$$\tilde{C}_{\text{fed}}(u, v) = \frac{\sum_{i=1}^K M_i(u, v) \tilde{C}_i(u, v)}{\sum_{i=1}^K M_i(u, v)},$$

siempre que al menos un cliente haya observado el par. Si ningún cliente aporta información sobre (u, v) , se asigna de nuevo el valor neutro 0.5. En esta agregación se utiliza M_i como conteo real, ya que representa cuánta evidencia aporta cada cliente sobre cada comparación.

Finalmente, el servidor resuelve el OBOP sobre $\tilde{C}_{\text{fed}}$ y obtiene el *bucket order* federado $\tilde{\beta}_{\text{fed}}$, cuya matriz asociada se denota por $\tilde{B}_{\text{fed}}$. Este constituye el resultado global de la simulación.

El procedimiento completo se resume en el Algoritmo 3.5.

Algoritmo 3.5: Procedimiento de simulación federada.

Entrada: Colección de rankings R , número de clientes K , parámetro de ruido b , técnica de perturbación y semilla aleatoria

Salida: *Bucket order* federado $\tilde{\beta}_{\text{fed}}$

- 1: Inicializar los generadores aleatorios con la semilla indicada
 - 2: Dividir R aleatoriamente en K subconjuntos $R^{(1)}, \dots, R^{(K)}$
 - 3: **para** cada cliente $i = 1, \dots, K$ **hacer**
 - 4: Construir W_i , M_i y C_i
 - 5: Construir la máscara P_i a partir de M_i
 - 6: Aplicar ruido a C_i solo en pares observados y obtener $\tilde{C}_i$
 - 7: Enviar $\tilde{C}_i$ y M_i al servidor
 - 8: El servidor construye $\tilde{C}_{\text{fed}}$ agregando las matrices locales con M_i
 - 9: Resolver el OBOP sobre $\tilde{C}_{\text{fed}}$
 - 10: **devolver** $\tilde{\beta}_{\text{fed}}$
-

3.4. Métricas de evaluación

Una vez definidas las estrategias de introducción de ruido y la simulación federada, se describen las métricas utilizadas para evaluar los resultados. Estas métricas permiten analizar tres aspectos complementarios: la similitud entre *bucket orders*, la pérdida de calidad respecto a la matriz original y el nivel real de ruido introducido. En el caso federado, la evaluación principal estudia el comportamiento del *bucket order* federado frente al centralizado, mientras que las métricas locales se emplean únicamente como análisis complementario.

3.4.1. Coeficiente de correlación de Kendall τ_b

La principal métrica utilizada para comparar dos *bucket orders* es el coeficiente de correlación de Kendall τ_b . Esta medida resulta adecuada en este trabajo porque permite comparar rankings con empates [20].

Para calcularla, se consideran todos los pares de alternativas y se compara la relación que presentan en β^* y en $\tilde{\beta}$. Denotamos por n_c el número de pares concordantes y por n_d el número de pares discordantes. Además, como pueden existir empates, se tienen en cuenta las cantidades t_{β^*} y $t_{\tilde{\beta}}$, donde t_{β^*} representa el número de pares empatados en β^* pero no en $\tilde{\beta}$, y $t_{\tilde{\beta}}$ representa el número de pares empatados en $\tilde{\beta}$ pero no en β^* .

Entonces, el coeficiente de correlación de Kendall τ_b se define como:

$$\tau_b(\beta^*, \tilde{\beta}) = \frac{n_c - n_d}{\sqrt{(n_c + n_d + t_{\beta^*})(n_c + n_d + t_{\tilde{\beta}})}}. \quad (3.8)$$

El valor de τ_b pertenece al intervalo $[-1, 1]$. Un valor $\tau_b = 1$ indica acuerdo total entre ambos *bucket orders*; un valor $\tau_b = -1$ indica desacuerdo total; y un valor próximo a 0 indica que no existe una relación clara entre ambos. Por tanto, cuanto mayor sea el valor de τ_b , más estable se considera el *bucket order* obtenido tras introducir ruido.

3.4.2. Pérdida de calidad respecto a la matriz original

Además de comparar la similitud estructural entre β^* y $\tilde{\beta}$, interesa analizar si el *bucket order* obtenido tras introducir ruido sigue siendo una buena solución para la matriz original C .

Para medir la pérdida de calidad producida por la introducción de ruido, se evalúa

la matriz $\tilde{B}$, asociada al *bucket order* con ruido $\tilde{\beta}$ sobre la matriz original C , obteniendo el valor $D(\tilde{B}, C)$. Este valor se compara con $D(B^*, C)$, que corresponde al valor objetivo del *bucket order* original β^* sobre la misma matriz. A partir de ambos valores, la pérdida de calidad se define como:

$$\rho = \frac{D(\tilde{B}, C) - D(B^*, C)}{n(n-1)}. \quad (3.9)$$

Esta métrica se normaliza dividiendo entre $n(n-1)$, que corresponde al número de entradas situadas fuera de la diagonal principal cuando se cuentan ambas direcciones de cada par de alternativas. Aunque solo existen $n(n-1)/2$ pares independientes, la distancia D se calcula sobre la matriz completa y, por tanto, incluye tanto la entrada (u, v) como (v, u) . Si ρ es cercano a 0, el *bucket order* obtenido tras introducir ruido sigue ajustándose de forma similar a la matriz original. En cambio, valores mayores de 0 indican que la calidad respecto a los datos originales se ha degradado.

La pérdida de calidad debe interpretarse junto con la distancia de entrada y las métricas que comparan ambos *bucket orders*. El OBOP puede admitir varias soluciones óptimas para una misma matriz de precedencias, por lo que es posible obtener $\rho = 0$ aunque $\tilde{\beta} \neq \beta^*$, si ambas soluciones alcanzan el mismo valor objetivo sobre C . En estos casos, el coeficiente τ_b y la distancia de salida permiten identificar diferencias estructurales que la pérdida de calidad no refleja. Asimismo, la distancia de entrada permite comprobar si una pérdida reducida se debe a la estabilidad del resultado o a que la matriz apenas ha sido modificada. Por ello, estas métricas deben analizarse conjuntamente.

3.4.3. Distancia de entrada, distancia de salida y sensibilidad

Además de analizar la calidad y la similitud de los *bucket orders*, interesa evaluar tanto la magnitud de la perturbación introducida en la matriz de entrada como su efecto sobre el resultado del OBOP. El objetivo es modificar la información original sin degradar en exceso el *bucket order* obtenido. Para ello, se definen una distancia de entrada, una distancia de salida y una medida de sensibilidad relativa que relaciona ambas magnitudes.

La distancia de entrada mide cuánto se ha modificado la matriz de precedencias original C para obtener la matriz perturbada $\tilde{C}$. Se calcula como la diferencia absoluta media entre sus entradas fuera de la diagonal:

$$d_{\text{entrada}}(C, \tilde{C}) = \frac{\sum_{u \neq v} |C(u, v) - \tilde{C}(u, v)|}{n(n-1)}. \quad (3.10)$$

Esta métrica toma valores en el intervalo $[0, 1]$ y mide el grado medio de modificación introducido sobre las comparaciones por pares. Un valor cercano a cero indica que la matriz con ruido es muy parecida a la matriz original, mientras que un valor más alto indica que las comparaciones han cambiado en mayor medida.

Esta distancia permite cuantificar el ruido introducido realmente, puesto que un mismo valor del parámetro b puede producir perturbaciones diferentes en función de la técnica, la instancia y la semilla aleatoria. No obstante, se trata de una medida empírica de modificación de los datos y no constituye una garantía formal de privacidad.

La distancia de salida mide el cambio producido en el *bucket order* como consecuencia de la introducción de ruido. Para calcularla, se emplea la representación matricial B_β definida en el Capítulo 2. Sean B_{β^*} y $B_{\tilde{\beta}}$ las matrices asociadas, respectivamente, al *bucket order* original β^* y al obtenido sobre la matriz perturbada, $\tilde{\beta}$. La distancia de salida se calcula aplicando sobre estas matrices la misma distancia normalizada utilizada para las matrices de precedencias:

$$d_{\text{salida}}(\beta^*, \tilde{\beta}) = \frac{\sum_{u \neq v} |B_{\beta^*}(u, v) - B_{\tilde{\beta}}(u, v)|}{n(n-1)}. \quad (3.11)$$

Esta métrica también toma valores en el intervalo $[0, 1]$. Si $d_{\text{salida}} = 0$, ambos *bucket orders* establecen las mismas relaciones de precedencia y empate. A medida que aumenta su valor, también aumenta la diferencia media entre sus relaciones por pares.

La distancia de salida complementa al coeficiente de Kendall τ_b . Mientras que τ_b expresa el grado de concordancia entre los dos *bucket orders*, la distancia de salida cuantifica directamente la magnitud media de los cambios producidos en sus relaciones por pares.

Finalmente, se define la sensibilidad relativa al ruido como el cociente entre la distancia de salida y la distancia de entrada:

$$S = \frac{d_{\text{salida}}(\beta^*, \tilde{\beta})}{d_{\text{entrada}}(C, \tilde{C})}. \quad (3.12)$$

Esta métrica expresa cuánto cambia el *bucket order* en relación con la modificación introducida en la matriz de precedencias. No mide la cantidad absoluta de ruido ni la pérdida de calidad de la solución, sino la respuesta relativa del resultado ante el ruido

aplicado.

- Si $S < 1$, el cambio observado en la salida es menor que el introducido en la entrada.
- Si $S \approx 1$, las distancias normalizadas de entrada y salida tienen una magnitud similar.
- Si $S > 1$, el cambio observado en la salida es mayor que el introducido en la entrada.

Por tanto, valores bajos de S indican una mayor estabilidad del resultado ante la perturbación considerada, mientras que valores más elevados indican una mayor sensibilidad. Esta medida es empírica, no está acotada superiormente y no debe confundirse con la noción formal de sensibilidad utilizada en privacidad diferencial.

3.4.4. Número de *buckets*

Como complemento a las métricas anteriores, también se registra el número de *buckets* en los *bucket orders* obtenidos. Esta información permite interpretar mejor algunos cambios producidos por el ruido. En particular, ayuda a detectar si una estrategia tiende a romper empates, crear nuevos empates o modificar la estructura general del *bucket order*.

3.4.5. Métricas en la simulación federada

En la simulación federada, la evaluación principal compara el *bucket order* federado $\tilde{\beta}_{\text{fed}}$ con el resultado centralizado de referencia β^* . Para ello, se utilizan las métricas definidas en la Sección 3.4: el coeficiente de Kendall τ_b , la distancia de entrada, la distancia de salida y la sensibilidad. En este caso, τ_b compara β^* y $\tilde{\beta}_{\text{fed}}$, la distancia de entrada compara C y $\tilde{C}_{\text{fed}}$, y la distancia de salida compara las matrices asociadas a ambos *bucket orders*.

Como análisis complementario, también se calculan métricas locales. Para disponer de una referencia local, se obtiene el *bucket order* β_i de cada cliente considerando únicamente los pares observados:

$$\beta_i = \arg \min_{\beta} \sum_{u \neq v} P_i(u, v) |B_{\beta}(u, v) - C_i(u, v)|.$$

Este resultado se calcula únicamente con fines descriptivos: no interviene en la agregación ni se envía al servidor. La pérdida local ρ_i compara el ajuste del resultado federado respecto a la matriz C_i con el alcanzado por β_i , mientras que $d_{\text{entrada},i}$ mide el cambio introducido en la matriz local al añadir ruido. Ambas métricas se calculan sobre los pares observados mediante la máscara P_i .

El *bucket order* federado se obtiene agregando la información de todos los clientes, por lo que no tiene por qué coincidir con el resultado que cada uno obtendría utilizando únicamente sus propios rankings. Esta diferencia es esperable y no implica por sí sola un peor funcionamiento del procedimiento federado. Por ello, las métricas locales se utilizan únicamente para contextualizar las diferencias entre la información local y el resultado global. La evaluación principal se centra en comparar el *bucket order* federado con el resultado centralizado de referencia.

3.5. Metodología seguida durante el desarrollo del trabajo

El desarrollo de este Trabajo Fin de Grado se ha llevado a cabo siguiendo un enfoque iterativo e incremental, adecuado para un proyecto de carácter experimental. En lugar de partir de una planificación completamente cerrada, el trabajo se organizó en fases sucesivas, en las que se implementaban nuevas funcionalidades, se validaban los resultados obtenidos y se ajustaban las siguientes decisiones experimentales.

En una primera fase se realizó el estudio bibliográfico sobre agregación de rankings, *bucket orders*, OBOP, privacidad y aprendizaje federado. A partir de esta base, se implementó el flujo centralizado inicial: lectura de instancias en formato PrefLib, construcción de la matriz de precedencias C , resolución exacta del OBOP mediante Programación Lineal Entera y obtención del *bucket order* de referencia.

Posteriormente, se incorporaron de forma progresiva las estrategias de introducción de ruido. Primero se implementó la introducción de ruido directa sobre la matriz de precedencias; después, la introducción de ruido sobre rankings individuales; y, finalmente, la estrategia basada en *scores* latentes y reconstrucción de la matriz. Cada estrategia fue probada de forma independiente antes de integrarse en el procedimiento experimental general.

Una vez validado el flujo centralizado con ruido, se añadieron las métricas de evaluación y se automatizó la ejecución de experimentos para distintas instancias, técnicas, valores de b y semillas aleatorias. Finalmente, se implementó la simulación federada por matrices locales y se almacenaron los resultados obtenidos en ficheros CSV para

su posterior análisis.

En cuanto a los roles, la autora ha sido la encargada de realizar el estudio bibliográfico, diseñar e implementar el entorno experimental, ejecutar los experimentos, interpretar los resultados y redactar la memoria. Los tutores han desempeñado un papel de supervisión y orientación, revisando el trabajo realizado, proponiendo ajustes metodológicos y ayudando a enfocar la experimentación.

Los detalles técnicos de la implementación, la estructura del repositorio y el procedimiento de ejecución de los experimentos se recogen en el Anexo A, mientras que los recursos software y hardware empleados se describen en el Anexo C.

4. Experimentación y análisis de resultados

En este capítulo se presenta la evaluación experimental del entorno desarrollado. El objetivo principal es analizar cómo varía el *bucket order* óptimo del OBOP cuando se introducen perturbaciones sobre distintas representaciones de las preferencias: la matriz de precedencias C , los rankings individuales y una representación intermedia basada en *scores* latentes. Además, se incorpora una simulación federada con el fin de estudiar si el procedimiento puede adaptarse a un escenario distribuido en el que los rankings no se centralizan directamente.

A diferencia del capítulo anterior, donde se describieron las estrategias implementadas y las métricas utilizadas, en este capítulo el foco se sitúa en los resultados obtenidos. Para ello, se parte de un *baseline* centralizado sin ruido que proporciona el *bucket order* óptimo de referencia para cada instancia. Posteriormente, se comparan los *bucket orders* obtenidos tras introducir ruido en el escenario centralizado y, finalmente, se analiza el comportamiento y la calidad de los resultados de la aproximación federada propuesta.

4.1. Diseño experimental

La evaluación experimental se ha llevado a cabo mediante el entorno desarrollado en Python. La implementación se organiza en distintos módulos: lectura y transformación de datos, construcción de la matriz C , resolución exacta del OBOP, generación de ruido, simulación federada y cálculo de métricas.

En el cuerpo principal de este capítulo se describen únicamente los aspectos necesarios para interpretar los resultados experimentales. Los detalles técnicos de implementación, la organización del repositorio y los scripts utilizados se recogen en el

4.1.1. Conjuntos de datos

Para la evaluación experimental se han utilizado distintas colecciones procedentes de PrefLib¹, una librería pública de datos de preferencias que proporciona instancias en distintos formatos ordinales [21]. Esta variedad permite evaluar el comportamiento del sistema ante rankings completos, incompletos, estrictos y con empates.

En concreto, se han considerado los siguientes formatos:

- **.soc**: rankings completos y estrictos.
- **.soi**: rankings estrictos que pueden ser incompletos.
- **.toc**: rankings completos que permiten empates.
- **.toi**: rankings incompletos que permiten empates.

En este contexto, una *colección* agrupa un conjunto de instancias relacionadas con un mismo dominio, mientras que una *instancia*² corresponde a un fichero concreto que contiene un conjunto de rankings definidos sobre un mismo conjunto de alternativas.

Cada instancia se caracteriza principalmente por dos parámetros:

- El número de alternativas n , que representa la cantidad de elementos que pueden aparecer en los rankings.
- El número de rankings m , que representa la cantidad de preferencias individuales (o votantes) recogidas en la instancia.

En todos los casos, las instancias se transforman en una matriz de precedencias C , que constituye la entrada del OBOP. El procedimiento detallado de construcción de dicha matriz y el tratamiento de las multiplicidades se describen en el Anexo A.

Dado que en este trabajo el *bucket order* se obtiene resolviendo el OBOP mediante ILP, el tamaño de las instancias tiene un impacto directo sobre el tiempo de resolución. A medida que aumenta el número de alternativas n , crece exponencialmente el espacio de búsqueda de posibles *bucket orders*, por lo que el tiempo necesario para resolver el problema de forma exacta puede aumentar significativamente.

¹<https://preflib.org/>

²En este contexto, el término *instancia* equivale a un fichero o *dataset* concreto de PrefLib, formado por una colección de rankings sobre las mismas alternativas.

Por este motivo, se han considerado principalmente instancias de tamaño reducido o moderado, con el fin de poder resolver el OBOP y repetir el proceso para diferentes estrategias de perturbación, valores del parámetro de ruido b y semillas aleatorias. La Tabla 4.1 resume las colecciones consideradas y las instancias seleccionadas de cada una.

Colección	Instancia seleccionada	Formato	n	m	Dominio
Skate Data	00006-00000040	toc	24	9	Clasificaciones de competiciones de patinaje.
Breakfast	00035-00000006	soc	15	42	Preferencias individuales sobre opciones de desayuno.
Poland Local Elections	00068-00000301	soi	15	15	Preferencias electorales en elecciones locales polacas.
Voter Autrement	00071-00000033	toi	11	1302	Datos de votación del proyecto Voter Autrement.

Tabla 4.1: Colecciones e instancias seleccionadas para la evaluación experimental.

Estas instancias permiten estudiar situaciones diferentes, tanto por el formato de los rankings de entrada como por el número de alternativas y el número de rankings.

4.2. Baseline centralizado sin ruido

En el primer bloque experimental se resuelve el OBOP mediante ILP sobre las instancias seleccionadas, sin aplicar ninguna perturbación previa. Este bloque proporciona, para cada instancia, el *bucket order* óptimo de referencia β^* y permite caracterizar estructuralmente las instancias antes de introducir ruido e implementar el entorno federado.

En esta fase no interviene ningún componente aleatorio, por lo que cada instancia se ejecuta una única vez. La Tabla 4.2 recoge, para cada instancia, el número de *buckets* y el *bucket order* óptimo de referencia β^* .

Los resultados muestran diferencias en la estructura de los *bucket orders*, entendida como el número de *buckets* y la distribución de las alternativas entre ellos. La

Instancia	Nº Buckets	β^*
00006-000000040	9	$\{23\} \mid \{21\} \mid \{20, 22, 24\} \mid \{18, 19\} \mid \{14, 17\} \mid \{7, 13, 15\} \mid \{3, 4, 5, 6, 12\} \mid \{8, 9, 10, 16\} \mid \{1, 2, 11\}$
00035-000000006	2	$\{12, 14\} \mid \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 13, 15\}$
00068-000000301	5	$\{7\} \mid \{12\} \mid \{8, 11, 15\} \mid \{1, 2, 3, 5, 6, 9, 10, 13, 14\} \mid \{4\}$
00071-000000033	2	$\{3, 4, 9\} \mid \{1, 2, 5, 6, 7, 8, 10, 11\}$

Tabla 4.2: Resultados del baseline centralizado: instancias, número de buckets y bucket order.

instancia 00006-000000040 presenta la estructura más compleja, con nueve *buckets*, mientras que 00068-000000301 ocupa una posición intermedia, con cinco. Por su parte, 00035-000000006 y 00071-000000033 presentan estructuras más agrupadas, formadas únicamente por dos *buckets*. En estos casos, resulta especialmente interesante analizar si esta estructura se conserva al introducir ruido o si ambos *buckets* terminan fusionándose en uno solo.

Esta variedad permite estudiar el efecto del ruido sobre resultados con distintas estructuras de partida. En el bloque siguiente se analiza cómo varía el *bucket order* de cada instancia al aplicar las estrategias de perturbación propuestas en el Capítulo 3.

4.3. Experimentos centralizados con ruido

Este bloque experimental evalúa el impacto de inyectar ruido controlado sobre las preferencias utilizando las tres estrategias de introducción de ruido definidas previamente. El objetivo no es únicamente comprobar si el *bucket order* cambia, sino analizar cuánto ruido se introduce realmente, qué pérdida de calidad produce y cómo se modifica la estructura de *buckets*.

4.3.1. Efecto del parámetro de ruido b

En primer lugar, se analiza el efecto del parámetro b en cada una de las estrategias implementadas. Este análisis es necesario porque el mismo valor de b no produce el mismo grado de perturbación en todas las estrategias. En el ruido sobre matriz, b controla la escala del ruido añadido directamente a C ; en el ruido sobre rankings, actúa como probabilidad de modificar rankings individuales; y en la estrategia de introducir

ruido sobre *scores* latentes, afecta a una representación intermedia desde la que se reconstruye la matriz.

Para interpretar correctamente los resultados, se analiza la distancia de entrada d_{entrada} , que mide cuánto se aleja la matriz perturbada $\tilde{C}$ de la matriz original C . Esta métrica permite comparar el grado de modificación que recibe la entrada del OBOP, aunque no constituye una garantía formal de privacidad ni mide directamente el cambio aplicado sobre cada ranking individual.

La Figura 4.1 representa, para cada estrategia y valor de b , la media de d_{entrada} obtenida sobre las cuatro instancias y las tres semillas empleadas. Las bandas sombreadas muestran una desviación estándar alrededor de la media y reflejan conjuntamente la variabilidad debida a la instancia y a la realización aleatoria del ruido.

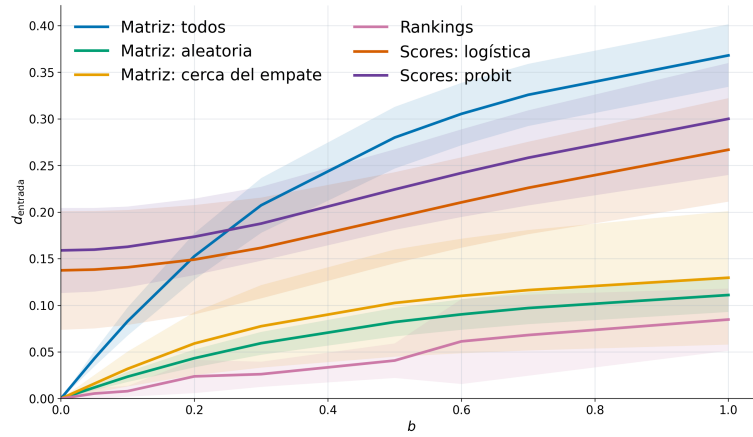


Figura 4.1: Efecto del parámetro b sobre la distancia de entrada para las estrategias de introducción de ruido evaluadas para todas las instancias.

Como se observa en la figura, d_{entrada} aumenta con b en todas las estrategias, aunque tanto la magnitud de la perturbación como su ritmo de crecimiento difieren considerablemente. La técnica **matriz (todos)** alcanza las mayores distancias de entrada. Este comportamiento es coherente con su funcionamiento, puesto que aplica ruido a todas las comparaciones por pares de la matriz.

Las técnicas **matriz (aleatoria)** y **matriz (cerca_empate)** presentan un crecimiento más moderado, debido a que actúan únicamente sobre un subconjunto de comparaciones. En las configuraciones analizadas, **cerca_empate** produce distancias medias ligeramente superiores a **aleatoria**, aunque también muestra una mayor dis-

persión entre ejecuciones.

La perturbación sobre rankings genera las menores modificaciones de la matriz agregada. Incluso para $b = 1$, su distancia de entrada media permanece por debajo de 0.10. Este resultado sugiere que parte de los cambios realizados sobre rankings individuales puede compensarse durante la construcción de $\tilde{C}$.

Las estrategias basadas en *scores* latentes presentan un comportamiento diferente. En ambos casos existe una distancia de entrada apreciable incluso cuando $b = 0$. Por tanto, la distancia observada no procede únicamente del ruido añadido, sino también de la reconstrucción de la matriz a partir de los *scores*. Al aumentar b , la distancia crece progresivamente y, en promedio, la variante **probit** introduce una modificación mayor que **logistic**.

Estos resultados confirman que un mismo valor de b no representa un nivel de perturbación equivalente entre estrategias. Por ello, en los análisis posteriores b se utiliza para identificar y reproducir las configuraciones experimentales, mientras que d_{entrada} se emplea como referencia principal para comparar la perturbación efectiva. Esta distinción permite interpretar los resultados sin asumir que configuraciones con el mismo valor de b son directamente comparables.

4.3.2. Análisis detallado de las instancias

Tras estudiar globalmente la relación entre el parámetro de ruido b y la distancia de entrada d_{entrada} , se analizan individualmente las cuatro instancias seleccionadas. Este análisis permite observar cómo influyen su tamaño, formato y estructura de preferencias en el comportamiento de las diferentes estrategias de ruido.

Para cada instancia se consideran distintas estrategias, valores de b y semillas aleatorias. Los valores de b se seleccionan con el objetivo de estudiar la evolución de cada estrategia dentro de su propio rango de funcionamiento, sin interpretarlos como niveles de perturbación equivalentes entre técnicas.

Los resultados se comparan con el *baseline* centralizado mediante la distancia de entrada d_{entrada} , el coeficiente de Kendall τ_b , la pérdida de calidad ρ , la sensibilidad al ruido S y el número de *buckets*.

Instancia 00006-00000040.toc

La instancia 00006-00000040.toc pertenece a la colección *Skate Data* de PrefLib y contiene rankings completos con empates. Está formada por $n = 24$ alternativas y

$m = 9$ rankings, y como muestra la Tabla 4.2, su *bucket order* de referencia contiene nueve *buckets*.

Esta instancia se incorpora al análisis detallado porque es la de mayor tamaño y presenta el resultado más fragmentado de las cuatro instancias seleccionadas.

En las tablas se muestran las ejecuciones correspondientes a las semillas 0, 1 y 2. Para cada técnica se seleccionan valores de b que representan niveles crecientes de ruido dentro de su propio rango de funcionamiento.

Técnica	b	Semilla	d_{entrada}	d_{salida}	S	τ_b	ρ	Buckets
todos	0.05	0	0.038	0.022	0.566	0.976	0.002	9
		1	0.031	0.000	0.000	1.000	0.000	9
		2	0.033	0.000	0.000	1.000	0.000	9
	0.10	0	0.073	0.024	0.322	0.974	0.005	10
		1	0.061	0.000	0.000	1.000	0.000	9
		2	0.063	0.011	0.172	0.988	0.003	8
	0.20	0	0.137	0.042	0.303	0.955	0.019	13
		1	0.115	0.043	0.378	0.953	0.020	14
		2	0.117	0.049	0.419	0.945	0.024	8
aleatoria	0.20	0	0.036	0.000	0.000	1.000	0.000	9
		1	0.039	0.004	0.093	0.996	0.004	10
		2	0.035	0.025	0.723	0.972	0.002	8
	0.50	0	0.078	0.027	0.347	0.970	0.019	12
		1	0.077	0.047	0.613	0.944	0.014	10
		2	0.073	0.025	0.348	0.964	0.006	10
	1.00	0	0.118	0.040	0.339	0.950	0.034	16
		1	0.115	0.076	0.664	0.890	0.038	14
		2	0.105	0.082	0.775	0.875	0.056	17
cerca_empate	0.10	0	0.009	0.005	0.584	0.994	0.005	10
		1	0.006	0.038	6.364	0.958	0.004	9
		2	0.006	0.005	0.960	0.994	0.001	9
	0.50	0	0.025	0.038	1.521	0.960	0.026	18
		1	0.020	0.042	2.104	0.955	0.011	13
		2	0.021	0.038	1.830	0.959	0.015	14
	1.00	0	0.030	0.049	1.639	0.949	0.036	22
		1	0.025	0.045	1.808	0.952	0.024	17
		2	0.028	0.040	1.399	0.954	0.023	17

Tabla 4.3: Ejecuciones de la estrategia de introducción de ruido sobre matriz para la instancia 00006-00000040.toc.

La Tabla 4.3 muestra el efecto de introducir ruido directamente sobre la matriz de precedencias. En conjunto, la instancia presenta una elevada robustez: incluso cuando

la matriz se modifica de forma apreciable, el *bucket order* resultante permanece próximo al de referencia, con valores altos de τ_b y pérdidas ρ reducidas.

La técnica **todos** es la que introduce una mayor perturbación para valores moderados de b , ya que actúa sobre todos los pares de alternativas. Sin embargo, el impacto sobre la salida es limitado en la mayoría de ejecuciones, lo que indica que la estructura principal del *bucket order* se conserva. La técnica **aleatoria**, por su parte, necesita valores más altos de b para alcanzar distancias de entrada similares, al modificar solo una parte de las comparaciones. En este caso se aprecia una mayor variabilidad entre semillas, lo que muestra que no solo importa la cantidad de ruido introducido, sino también qué pares concretos se ven afectados.

La técnica **cerca_empate** presenta un comportamiento distinto. Aunque no alcanza distancias de entrada tan elevadas como las otras técnicas, produce valores de sensibilidad altos. Esto se debe a que actúa precisamente sobre comparaciones próximas a 0.5, que son las más relacionadas con la formación o ruptura de empates. Por ello, una modificación pequeña en la entrada puede provocar cambios proporcionales mayores en la estructura interna del *bucket order*. Aun así, la orientación general del orden se conserva en gran medida, como reflejan los valores elevados de τ_b .

b	Semilla	d_{entrada}	d_{salida}	S	τ_b	ρ	Buckets
0.30	0	0.034	0.031	0.911	0.966	0.004	9
	1	0.041	0.000	0.000	1.000	0.000	9
	2	0.058	0.045	0.781	0.949	0.014	8
0.60	0	0.102	0.043	0.426	0.951	0.011	7
	1	0.185	0.241	1.304	0.683	0.184	5
	2	0.089	0.056	0.633	0.936	0.018	7
1.00	0	0.109	0.080	0.731	0.908	0.040	8
	1	0.168	0.114	0.678	0.860	0.070	5
	2	0.122	0.127	1.038	0.845	0.070	5

Tabla 4.4: Ejecuciones de la estrategia de introducción de ruido sobre rankings para la instancia 00006-00000040.toc.

La Tabla 4.4 recoge el efecto de introducir ruido sobre los rankings individuales antes de construir la matriz de precedencias. Para valores bajos de b , el resultado se mantiene muy próximo al *bucket order* original. Esto sugiere que parte de las modificaciones realizadas sobre rankings concretos queda amortiguada al agregarse la información en una única matriz C .

Al aumentar b , la dependencia de la semilla se vuelve más visible. Algunas ejecuciones siguen manteniendo una calidad elevada, mientras que otras reducen notablemente el número de *buckets* y aumentan la pérdida. Además, incluso manteniendo la misma semilla, un valor mayor de b no conduce necesariamente a un peor resultado. Al incrementar b , no se amplifica el ruido aplicado anteriormente, sino que aumenta la probabilidad de modificar rankings o posiciones diferentes. Por tanto, un valor mayor de b incrementa la perturbación esperada, pero no garantiza necesariamente un peor resultado en cada ejecución individual.

Técnica	b	Semilla	d_{entrada}	d_{salida}	S	τ_b	ρ	Buckets
logistic	0.00	–	0.186	0.190	1.024	0.760	0.135	2
		0	0.210	0.196	0.932	0.753	0.141	3
	0.20	1	0.193	0.167	0.862	0.795	0.114	3
		2	0.181	0.199	1.101	0.747	0.139	2
	0.50	0	0.265	0.264	0.999	0.561	0.209	4
		1	0.224	0.176	0.786	0.762	0.123	5
		2	0.187	0.196	1.049	0.727	0.143	3
probit	0.00	–	0.106	0.063	0.601	0.927	0.034	4
		0	0.150	0.138	0.921	0.830	0.091	4
	0.20	1	0.124	0.063	0.512	0.927	0.034	4
		2	0.109	0.105	0.968	0.876	0.058	4
	0.50	0	0.235	0.255	1.085	0.546	0.199	6
		1	0.178	0.188	1.058	0.701	0.145	6
		2	0.137	0.123	0.899	0.824	0.088	5

Tabla 4.5: Ejecuciones de la estrategia de introducción de ruido sobre *scores* para la instancia 00006-00000040.toc.

La Tabla 4.5 recoge los resultados de introducir ruido sobre *scores* latentes, incluyendo también el caso $b = 0$, lo que permite separar el efecto de la reconstrucción mediante *scores* del efecto del ruido añadido posteriormente. En esta instancia, la reconstrucción ya modifica de forma apreciable la matriz incluso sin ruido, reduciendo mucho el número de *buckets*. Por tanto, parte de la degradación no se debe al ruido, sino al propio procedimiento de reconstrucción.

La variante **probit** se ajusta mejor que la variante **logistic** a esta instancia, ya que conserva mejor la estructura del *bucket order* de referencia. Al aumentar b , ambas variantes tienden a degradar la calidad, aunque con cierta variabilidad entre semillas. En conjunto, esta familia de métodos se comporta más como una reconstrucción apro-

ximada de la matriz que como una perturbación directa comparable a las estrategias anteriores.

Esta instancia 00006-00000040.toc muestra una elevada robustez frente al ruido. Aunque es la de mayor tamaño y parte de un *bucket order* bastante fragmentado, la estructura principal del resultado se conserva en gran medida. Los cambios observados afectan sobre todo al número de *buckets*, debido a fusiones o divisiones de agrupaciones, pero no alteran de forma severa la orientación general del orden.

Instancia 00035-00000006.soc

La instancia 00035-00000006.soc pertenece a la colección *Breakfast* de PrefLib y contiene rankings completos y estrictos. Está formada por $n = 15$ alternativas y $m = 42$ rankings. Su *bucket order* de referencia contiene únicamente dos *buckets*, por lo que se utiliza como contraste respecto a 00006-00000040.toc: en este caso, el interés principal es comprobar si el ruido conserva la separación entre ambos grupos o si provoca su fusión.

La Tabla 4.6 muestra que esta instancia es más sensible que la anterior. Con valores similares de d_{entrada} , algunas ejecuciones conservan exactamente el resultado original, mientras que otras fusionan los dos *buckets* o fragmentan la solución en más grupos. Esto indica que el efecto del ruido no depende solo de su intensidad global, sino también de qué relaciones concretas se modifican.

Cuando los dos *buckets* se fusionan, τ_b puede caer hasta 0 porque desaparece la única separación relevante del *bucket order* original. Sin embargo, la pérdida ρ puede seguir siendo reducida, ya que la mayor parte de las alternativas ya estaban agrupadas. En esta instancia, una pérdida reducida no implica que se haya conservado la estructura: si el objetivo es mantener la separación entre ambos grupos, el número de *buckets* y τ_b resultan especialmente relevantes.

El ruido sobre rankings confirma este comportamiento. Para perturbaciones moderadas, parte de los cambios individuales queda amortiguada al construir la matriz agregada. Sin embargo, cuando se modifican relaciones decisivas, pequeñas variaciones en la entrada pueden eliminar la separación entre los dos *buckets*, lo que da lugar a valores altos de sensibilidad S . Las ejecuciones individuales se recogen en la Tabla B.1.

La Tabla 4.7 presenta una diferencia respecto a 00006-00000040.toc. Mientras que en la anterior instancia **probit** conservaba mejor el resultado, en este caso **logistic** es la que mejor mantiene la estructura original. Esto se debe a que la reconstrucción mediante **probit** tiende a dividir más los *buckets* originales, lo que afecta más a esta

Técnica	b	Semilla	d_{entrada}	d_{salida}	S	τ_b	ρ	Buckets
todos	0.05	0	0.057	0.124	2.161	0.000	0.006	1
		1	0.047	0.000	0.000	1.000	0.000	2
		2	0.048	0.000	0.000	1.000	0.000	2
	0.10	0	0.111	0.124	1.116	0.000	0.006	1
		1	0.092	0.000	0.000	1.000	0.000	2
		2	0.094	0.000	0.000	1.000	0.000	2
	0.20	0	0.193	0.171	0.888	0.105	0.034	2
		1	0.173	0.124	0.716	0.000	0.006	1
		2	0.171	0.057	0.334	0.827	0.013	3
aleatoria	0.20	0	0.055	0.124	2.244	0.000	0.006	1
		1	0.045	0.124	2.762	0.000	0.006	1
		2	0.048	0.000	0.000	1.000	0.000	2
	0.50	0	0.107	0.124	1.158	0.000	0.006	1
		1	0.086	0.124	1.435	0.000	0.006	1
		2	0.083	0.000	0.000	1.000	0.000	2
	1.00	0	0.146	0.124	0.849	0.000	0.006	1
		1	0.115	0.129	1.120	0.453	0.035	3
		2	0.103	0.000	0.000	1.000	0.000	2
cerca_empate	0.10	0	0.039	0.000	0.000	1.000	0.000	2
		1	0.028	0.000	0.000	1.000	0.000	2
		2	0.024	0.000	0.000	1.000	0.000	2
	0.50	0	0.098	0.067	0.681	0.681	0.002	2
		1	0.090	0.000	0.000	1.000	0.000	2
		2	0.091	0.005	0.053	0.981	0.005	3
	1.00	0	0.121	0.119	0.984	0.700	0.073	4
		1	0.113	0.000	0.000	1.000	0.000	2
		2	0.117	0.062	0.531	0.816	0.018	4

Tabla 4.6: Ejecuciones de la estrategia de introducción de ruido sobre matriz para la instancia 00035-00000006.soc.

instancia porque su *bucket order* de referencia solo contiene dos *buckets*.

De manera general, esta instancia muestra que una solución formada por pocos *buckets* puede ser especialmente sensible a cambios localizados. Ejecuciones con distancias de entrada similares pueden producir resultados muy diferentes según las relaciones de pares afectadas.

Técnica	b	Semilla	d_{entrada}	d_{salida}	S	τ_b	ρ	Buckets
logistic	0.00	–	0.088	0.143	1.631	0.681	0.043	3
		0	0.106	0.210	1.984	0.480	0.060	2
	0.20	1	0.111	0.190	1.714	0.628	0.064	3
		2	0.104	0.162	1.562	0.629	0.048	3
	0.50	0	0.205	0.367	1.786	-0.049	0.163	3
		1	0.162	0.229	1.410	0.593	0.107	4
		2	0.137	0.219	1.602	0.523	0.078	3
probit	0.00	–	0.165	0.243	1.469	0.581	0.093	5
		0	0.178	0.276	1.550	0.428	0.086	3
	0.20	1	0.185	0.248	1.339	0.577	0.104	4
		2	0.180	0.248	1.373	0.577	0.097	4
	0.50	0	0.276	0.433	1.568	0.000	0.217	5
		1	0.236	0.290	1.229	0.547	0.148	6
		2	0.212	0.276	1.300	0.526	0.109	4

Tabla 4.7: Ejecuciones de la estrategia de introducción de ruido sobre *scores* para la instancia 00035-00000006.soc.

Instancia 00068-00000301.soi

La instancia 00068-00000301.soi pertenece a la colección *Poland Local Elections* de PrefLib y contiene rankings estrictos incompletos. Está formada por $n = 15$ alternativas y $m = 15$ rankings, y su *bucket order* de referencia contiene cinco *buckets*.

Esta estructura intermedia permite estudiar un caso distinto a los anteriores: no es un resultado tan fragmentado como 00006-00000040.toc, pero tampoco una solución formada casi por un único grupo como 00035-00000006.soc.

Los resultados del ruido sobre matriz mantienen la dependencia respecto a los pares perturbados observada anteriormente. Las estrategias basadas en *scores* también muestran el efecto de la reconstrucción inicial, con mejores resultados para **logistic** que para **probit**. Las tablas completas se recogen en las Tablas B.2 y B.3.

El comportamiento más representativo de esta instancia aparece al introducir ruido sobre los rankings individuales.

La Tabla 4.8 muestra que, para valores bajos y medios de b , la mayoría de ejecuciones conserva exactamente el *bucket order* de referencia. Sin embargo, algunas semillas modifican la estructura incluso con distancias de entrada muy reducidas. El caso más claro aparece con $b = 0.10$, donde una distancia de entrada prácticamente nula produce un cambio en el número de *buckets* y un valor de sensibilidad muy alto.

Este valor de S no debe interpretarse como una degradación extrema del resultado,

b	Semilla	d_{entrada}	d_{salida}	S	τ_b	ρ	Buckets
0.10	0	0.014	0.000	0.000	1.000	0.000	5
	1	0.010	0.000	0.000	1.000	0.000	5
	2	0.002	0.100	51.882	0.826	0.002	4
0.50	0	0.023	0.000	0.000	1.000	0.000	5
	1	0.051	0.000	0.000	1.000	0.000	5
	2	0.035	0.100	2.895	0.826	0.002	4
1.00	0	0.092	0.252	2.748	0.379	0.059	3
	1	0.070	0.214	3.073	0.545	0.033	3
	2	0.056	0.043	0.771	0.929	0.005	4

Tabla 4.8: Ejecuciones de la estrategia de introducción de ruido sobre rankings para la instancia 00068-00000301.soi.

sino como consecuencia de dividir una distancia de salida moderada entre una distancia de entrada muy pequeña. La pérdida ρ sigue siendo reducida, lo que indica que el nuevo *bucket order* apenas empeora el valor objetivo, aunque sí cambia parte de su estructura.

Esta instancia muestra que el ruido aplicado sobre rankings individuales puede quedar amortiguado al construir la matriz agregada, pero también puede alterar el resultado cuando afecta a relaciones poco representadas. Al haber solo 15 rankings incompletos, la semilla adquiere más importancia: no basta con medir cuánto cambia la matriz, sino qué relaciones concretas se han visto modificadas.

Instancia 00071-00000033.toi

La instancia 00071-00000033.toi pertenece a la colección *Voter Autrement* de PrefLib y contiene rankings incompletos con empates. Está formada por $n = 11$ alternativas y $m = 1302$ rankings, y su *bucket order* de referencia contiene dos *buckets*. Esta instancia permite contrastar el caso 00035-00000006.soc, ya que también tiene una estructura de dos grupos, pero se construye a partir de un número mucho mayor de preferencias individuales.

Técnica	b	Semilla	d_{entrada}	d_{salida}	S	τ_b	ρ	Buckets
todos	0.05	0	0.054	0.000	0.000	1.000	0.000	2
		1	0.044	0.000	0.000	1.000	0.000	2
		2	0.039	0.000	0.000	1.000	0.000	2
	0.10	0	0.097	0.000	0.000	1.000	0.000	2
		1	0.084	0.000	0.000	1.000	0.000	2
		2	0.077	0.000	0.000	1.000	0.000	2
	0.20	0	0.173	0.091	0.524	0.770	0.021	2
		1	0.157	0.091	0.579	0.840	0.060	5
		2	0.140	0.000	0.000	1.000	0.000	2
aleatoria	0.20	0	0.064	0.000	0.000	1.000	0.000	2
		1	0.047	0.000	0.000	1.000	0.000	2
		2	0.043	0.000	0.000	1.000	0.000	2
	0.50	0	0.111	0.000	0.000	1.000	0.000	2
		1	0.089	0.000	0.000	1.000	0.000	2
		2	0.069	0.091	1.308	0.810	0.031	2
	1.00	0	0.133	0.000	0.000	1.000	0.000	2
		1	0.102	0.000	0.000	1.000	0.000	2
		2	0.091	0.091	1.000	0.810	0.031	2
cerca_empate	0.10	0	0.056	0.000	0.000	1.000	0.000	2
		1	0.032	0.000	0.000	1.000	0.000	2
		2	0.029	0.000	0.000	1.000	0.000	2
	0.50	0	0.148	0.273	1.847	0.651	0.194	8
		1	0.117	0.182	1.560	0.695	0.112	3
		2	0.110	0.018	0.165	0.961	0.018	3
	1.00	0	0.173	0.264	1.523	0.657	0.199	8
		1	0.150	0.264	1.756	0.625	0.171	6
		2	0.156	0.264	1.690	0.673	0.202	9

Tabla 4.9: Ejecuciones de la estrategia de introducción de ruido sobre matriz para la instancia 00071-00000033.toi.

La Tabla 4.9 muestra una estabilidad mayor que la observada en 00035-00000006.soc. Para niveles bajos de perturbación, el *bucket order* de referencia se conserva en todas las ejecuciones, y solo aparecen cambios relevantes cuando aumenta el valor de b .

El comportamiento más variable aparece en la técnica `cerca_empate`, cuando el ruido se concentra en comparaciones próximas a 0.5. Para valores medios y altos de b , el resultado se fragmenta y aumentan d_{salida} y la pérdida. No obstante, al tratarse de una instancia incompleta, algunos valores próximos a 0.5 pueden representar ausencia de información y no una relación equilibrada. Por tanto, parte de los cambios podría deberse a la introducción de preferencias sobre pares poco observados o no observados.

Los resultados del ruido sobre rankings siguen la misma tendencia observada en las

instancias anteriores: para perturbaciones reducidas, los cambios quedan amortiguados durante la agregación, mientras que para valores mayores de b puede perderse la separación entre los dos *buckets*. Como no aportan un patrón nuevo respecto a los casos ya comentados, las ejecuciones completas se recogen en la Tabla B.4.

Técnica	b	Semilla	d_{entrada}	d_{salida}	S	τ_b	ρ	Buckets
logistic	0.00	–	0.068	0.000	0.000	1.000	0.000	2
		0	0.066	0.000	0.000	1.000	0.000	2
	0.20	1	0.099	0.000	0.000	1.000	0.000	2
		2	0.083	0.000	0.000	1.000	0.000	2
	0.50	0	0.154	0.209	1.356	0.468	0.073	3
		1	0.156	0.082	0.526	0.853	0.066	4
		2	0.120	0.209	1.744	0.638	0.109	4
probit	0.00	–	0.140	0.109	0.779	0.816	0.055	3
		0	0.137	0.191	1.391	0.701	0.097	4
	0.20	1	0.171	0.155	0.904	0.765	0.096	4
		2	0.162	0.155	0.956	0.765	0.088	4
	0.50	0	0.211	0.236	1.120	0.480	0.096	3
		1	0.223	0.191	0.856	0.730	0.146	5
		2	0.188	0.155	0.822	0.765	0.098	4

Tabla 4.10: Ejecuciones de la estrategia de introducción de ruido sobre *scores* para la instancia 00071-00000033.toi.

La estrategia basada en *scores* recoge sus resultados en la Tabla 4.10. Estos resultados muestran que una distancia de entrada apreciable no implica necesariamente un cambio en el *bucket order* final. En esta instancia, la reconstrucción puede modificar la matriz sin alterar el resultado, siempre que se conserven las relaciones que sostienen la separación entre los *buckets*. Por tanto, este caso aporta un ejemplo claro de robustez frente a la reconstrucción: el efecto de la perturbación no depende solo de cuánto cambia la matriz, sino de si se ven afectadas las comparaciones decisivas para la estructura del resultado.

4.3.3. Comparación global de las estrategias centralizadas

Una vez analizadas las instancias individualmente, se comparan las estrategias en función de la perturbación efectiva introducida. Para ello, las Figuras 4.2a–4.2c representan la distancia de salida, el coeficiente de Kendall y la pérdida de calidad frente a

d_{entrada} .

Para cada estrategia y valor de b , las líneas representan la media obtenida sobre las cuatro instancias y las tres semillas. Las bandas sombreadas muestran una desviación estándar de la métrica situada en el eje vertical, mientras que las barras horizontales representan una desviación estándar de d_{entrada} .

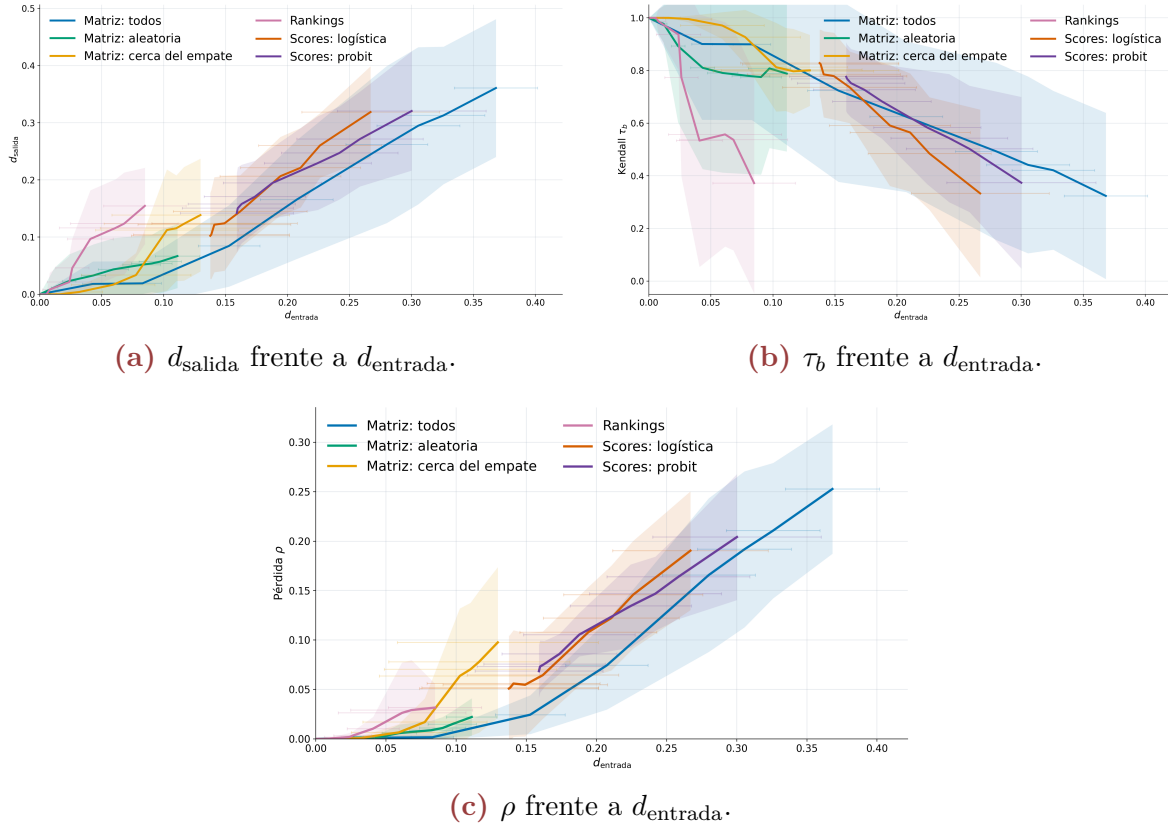


Figura 4.2: Relación entre la introducción de ruido efectiva de entrada y las métricas de salida en los experimentos centralizados. Las líneas representan los valores medios, mientras que las bandas sombreadas y las barras horizontales muestran una desviación estándar.

En términos generales, al aumentar d_{entrada} también aumentan la distancia de salida y la pérdida, mientras que el coeficiente τ_b disminuye. Sin embargo, las diferencias entre las trayectorias y la amplitud de las bandas muestran que una perturbación de entrada similar no produce necesariamente el mismo efecto en todas las estrategias ni en todas las instancias.

La técnica **todos** alcanza el mayor rango de perturbación y presenta una degrada-

ción progresiva. Las técnicas *aleatoria* y *cerca_empate* modifican una parte menor de la matriz, aunque esta última puede producir cambios estructurales apreciables al actuar sobre comparaciones próximas al empate. El ruido sobre rankings genera distancias de entrada reducidas, pero muestra una variabilidad elevada. Esto sugiere que su efecto depende de las preferencias concretas modificadas y de cómo estas se combinan al construir la matriz agregada. Por su parte, las técnicas basadas en *scores* presentan una perturbación positiva incluso para $b = 0$, debido al efecto de la reconstrucción de la matriz.

El análisis detallado muestra que la estructura inicial del *bucket order* condiciona la respuesta al ruido. En soluciones con pocos *buckets*, una fusión puede reducir considerablemente τ_b , aunque la pérdida respecto a la matriz original permanezca baja. Por el contrario, en resultados más fragmentados, el ruido puede crear o eliminar algunos *buckets* sin alterar por completo la orientación general del orden.

También influye la cantidad y distribución de la información disponible. En las instancias construidas a partir de pocos rankings o de preferencias incompletas, el resultado puede depender en mayor medida de las comparaciones con menor representación. En estos casos, ejecuciones con distancias de entrada parecidas pueden producir resultados muy diferentes según las relaciones modificadas.

Por tanto, no puede identificarse una estrategia superior para todos los casos ni evaluar el comportamiento únicamente mediante d_{entrada} . La comparación debe considerar conjuntamente τ_b , d_{salida} , ρ y el número de *buckets*, ya que cada métrica recoge un aspecto distinto de la conservación del *bucket order*.

4.4. Resultados de la simulación federada

En esta sección se analiza si el *bucket order* global obtenido en la simulación federada se mantiene próximo al resultado centralizado de referencia cuando el ruido se introduce en las matrices locales antes de agregarlas.

Los ficheros de salida contienen una fila por cliente, pero las métricas globales son comunes a todos los clientes de una ejecución. Por ello, se contabilizan una única vez por combinación de instancia, técnica, b , K^3 y semilla. Las métricas locales se presentan únicamente como información complementaria en el Anexo B.

Para simplificar las tablas, se utiliza el superíndice c para el escenario centralizado y f para el escenario federado.

³ K = número de clientes

4.4.1. Simulación federada sin ruido

En primer lugar, se comprueba que la simulación federada reproduce correctamente el comportamiento centralizado cuando no se introduce ruido. En esta configuración, cada cliente construye su matriz local a partir de los rankings que tiene asignados y la envía al servidor sin perturbar. Al agregarse las matrices locales de forma ponderada, el servidor debe reconstruir la misma información que se obtendría si todos los rankings se procesaran directamente en un único nodo.

Instancia	K	d_{entrada}	d_{salida}	τ_b
00006-000000040	3	0	0	1
00035-000000006	3	0	0	1
00068-00000301	3	0	0	1
00071-000000033	3	0	0	1

Tabla 4.11: Simulación federada de control con $b = 0$ y $K = 3$.

Como se observa en la Tabla 4.11, en todas las instancias se obtiene $d_{\text{entrada}} = 0$, $d_{\text{salida}} = 0$ y $\tau_b = 1$. Por tanto, repartir los rankings entre clientes y agregar posteriormente sus matrices locales no altera por sí solo el *bucket order* obtenido por el servidor.

4.4.2. Comparación entre centralizado y federado

Una vez validado el caso sin ruido, se compara el comportamiento del escenario federado con el escenario centralizado. El objetivo de este análisis es comprobar si el *bucket order* federado se mantiene próximo al *bucket order* centralizado de referencia cuando las matrices locales se perturban antes de ser enviadas al servidor.

Para que la comparación sea coherente, se utilizan las mismas técnicas de ruido sobre matriz y los mismos valores de b en ambos escenarios. En el escenario federado se fija el número de clientes en función del número de rankings disponibles en cada instancia: $K = 2$ para 00006-000000040 y 00068-00000301, y $K = 4$ para 00035-000000006 y 00071-000000033.

La Tabla 4.12 resume las métricas globales obtenidas. Cada fila corresponde a una instancia y una técnica de ruido, y recoge el promedio de las ejecuciones realizadas

con los valores de b seleccionados y las distintas semillas. Para construir este resumen se excluye el caso de control $b = 0$. En la técnica `todos` se consideran los valores $b \in \{0.05, 0.10, 0.20\}$, mientras que en `aleatoria` y `cerca_empate` se utilizan $b \in \{0.05, 0.10, 0.50, 1.00\}$. Cada configuración se repite con las semillas 0, 1 y 2.

Para comparar directamente ambos escenarios se calcula la diferencia entre los valores medios de τ_b :

$$\Delta\tau_b = \overline{\tau_b^f} - \overline{\tau_b^c},$$

donde $\overline{\tau_b^f}$ es el valor medio obtenido en el escenario federado y $\overline{\tau_b^c}$ es el valor medio obtenido en el escenario centralizado. En ambos casos, τ_b mide la similitud entre el *bucket order* obtenido tras introducir ruido y el *bucket order* de referencia sin ruido. Por tanto, valores positivos de $\Delta\tau_b$ indican que el resultado federado se mantiene más próximo a la referencia, mientras que valores negativos indican un mejor comportamiento del escenario centralizado.

Instancia	Técnica	K	τ_b^c/τ_b^f	$\Delta\tau_b$	$d_{\text{entrada}}^c/d_{\text{entrada}}^f$	$d_{\text{salida}}^c/d_{\text{salida}}^f$	S^c/S^f
00006-00000040	<code>aleatoria</code>	2	0.961/0.981	0.020	0.054/0.052	0.029/0.017	0.608/0.195
	<code>cerca_empate</code>	2	0.973/0.987	0.014	0.015/0.014	0.025/0.012	1.517/0.785
	<code>todos</code>	2	0.977/0.984	0.008	0.074/0.064	0.021/0.014	0.240/0.153
00035-00000006	<code>aleatoria</code>	4	0.621/0.473	-0.148	0.064/0.049	0.052/0.067	0.733/1.186
	<code>cerca_empate</code>	4	0.932/0.867	-0.064	0.064/0.054	0.021/0.028	0.187/0.440
	<code>todos</code>	4	0.548/0.631	0.083	0.110/0.058	0.067/0.049	0.579/0.755
00068-00000301	<code>aleatoria</code>	2	0.886/0.862	-0.024	0.051/0.016	0.058/0.069	1.919/9.818
	<code>cerca_empate</code>	2	0.849/0.984	0.135	0.116/0.002	0.104/0.010	0.551/3.294
	<code>todos</code>	2	0.884/0.719	-0.165	0.091/0.022	0.054/0.132	0.486/6.645
00071-00000033	<code>aleatoria</code>	4	0.968/0.750	-0.218	0.060/0.052	0.015/0.055	0.192/0.582
	<code>cerca_empate</code>	4	0.855/1.000	0.145	0.086/0.046	0.105/0.000	0.712/0.000
	<code>todos</code>	4	0.957/0.946	-0.010	0.096/0.053	0.020/0.018	0.123/0.200

Tabla 4.12: Resumen de las métricas globales en los escenarios centralizado y federado.

Los resultados muestran que el escenario federado se comporta, en general, de forma próxima al centralizado. Los valores de $\Delta\tau_b$ no presentan una tendencia uniforme: en algunas configuraciones son positivos, en otras negativos y en varios casos se mantienen cerca de cero. Por tanto, no se observa un deterioro sistemático por el hecho de introducir el ruido de forma local y agregar después las matrices en el servidor.

También se observa que d_{entrada}^f suele ser menor que d_{entrada}^c . Esto indica que, al agregar matrices locales perturbadas, parte de las modificaciones introducidas por los

clientes puede amortiguarse en la matriz federada final. No obstante, esta reducción de la distancia de entrada no garantiza siempre una mayor similitud del *bucket order*, ya que el efecto final depende de qué comparaciones concretas se hayan modificado.

Influencia del número de clientes

Por último, se analiza si variar el número de clientes K modifica el comportamiento global del *bucket order* federado. Al aumentar K , los rankings se dividen en más particiones y cada cliente trabaja con menos información local. Sin embargo, el objetivo de este análisis no es evaluar si el resultado representa mejor o peor a cada cliente individual, sino comprobar si el *bucket order* obtenido por el servidor sigue siendo próximo al resultado centralizado de referencia.

La Tabla 4.13 resume el cambio observado al pasar del menor al mayor valor de K ejecutado para cada configuración. Cada fila fija una instancia, una técnica de perturbación y un valor de b . Los valores mostrados corresponden a la media de las ejecuciones realizadas con las semillas 0, 1 y 2. La columna K indica el intervalo comparado; por ejemplo, 2 – 8 significa que se compara el valor medio obtenido con $K = 2$ frente al valor medio obtenido con $K = 8$. En las columnas de métricas, la notación $a \rightarrow b$ indica el valor obtenido con el menor K y el valor obtenido con el mayor K , respectivamente.

Instancia	Técnica	b	K	τ_b^f	d_{entrada}^f	d_{salida}^f	S^f
00006-000000040	todos	0.20	2-3	0.963 $\rightarrow$ 0.967	0.106 $\rightarrow$ 0.099	0.033 $\rightarrow$ 0.029	0.301 $\rightarrow$ 0.296
00006-000000040	aleatoria	0.50	2-3	0.964 $\rightarrow$ 0.995	0.074 $\rightarrow$ 0.067	0.033 $\rightarrow$ 0.005	0.443 $\rightarrow$ 0.077
00006-000000040	cerca_empate	0.50	2-3	0.980 $\rightarrow$ 1.000	0.021 $\rightarrow$ 0.000	0.019 $\rightarrow$ 0.000	0.841 $\rightarrow$ 0.000
00035-000000006	todos	0.05	2-8	0.894 $\rightarrow$ 0.894	0.037 $\rightarrow$ 0.019	0.022 $\rightarrow$ 0.022	0.588 $\rightarrow$ 1.100
00035-000000006	aleatoria	0.10	2-8	0.667 $\rightarrow$ 1.000	0.028 $\rightarrow$ 0.017	0.041 $\rightarrow$ 0.000	1.604 $\rightarrow$ 0.000
00035-000000006	cerca_empate	0.50	2-8	0.667 $\rightarrow$ 0.560	0.084 $\rightarrow$ 0.067	0.041 $\rightarrow$ 0.063	0.405 $\rightarrow$ 0.914
00068-000000301	todos	0.05	2-3	0.837 $\rightarrow$ 0.714	0.010 $\rightarrow$ 0.013	0.079 $\rightarrow$ 0.137	7.743 $\rightarrow$ 10.367
00068-000000301	aleatoria	0.10	2-3	0.861 $\rightarrow$ 0.861	0.005 $\rightarrow$ 0.008	0.073 $\rightarrow$ 0.073	14.087 $\rightarrow$ 8.436
00068-000000301	cerca_empate	0.50	2-3	0.969 $\rightarrow$ 1.000	0.003 $\rightarrow$ 0.004	0.019 $\rightarrow$ 0.000	6.719 $\rightarrow$ 0.000
00071-000000033	todos	0.20	2-8	1.000 $\rightarrow$ 1.000	0.126 $\rightarrow$ 0.062	0.000 $\rightarrow$ 0.000	0.000 $\rightarrow$ 0.000
00071-000000033	aleatoria	0.50	2-8	0.667 $\rightarrow$ 0.667	0.079 $\rightarrow$ 0.056	0.073 $\rightarrow$ 0.073	0.904 $\rightarrow$ 1.335
00071-000000033	cerca_empate	1.00	2-8	0.902 $\rightarrow$ 1.000	0.110 $\rightarrow$ 0.049	0.058 $\rightarrow$ 0.000	0.504 $\rightarrow$ 0.000

Tabla 4.13: Influencia del número de clientes sobre la simulación federada.

Al aumentar K , cada cliente construye su matriz local a partir de un menor número

de rankings. Los resultados no muestran una respuesta uniforme: en varias configuraciones aumenta el coeficiente τ_b y disminuyen la distancia de salida y la sensibilidad, mientras que en otras el resultado se mantiene o se degrada. Parte de las mejoras coincide con una reducción de la perturbación efectiva en la matriz federada, por lo que deben interpretarse conjuntamente d_{entrada}^f , d_{salida}^f y S^f . No obstante, también se observan configuraciones en las que la salida mejora aunque la distancia de entrada se mantenga o aumente, mostrando que la agregación puede compensar perturbaciones locales.

De manera conjunta, la simulación federada obtiene resultados globales generalmente próximos a los del escenario centralizado, aunque su comportamiento depende de la instancia, la técnica, el nivel de ruido y el número de clientes. Por tanto, dentro de las configuraciones estudiadas, el enfoque federado permite reducir la centralización directa de los rankings originales sin producir un deterioro sistemático del *bucket order* global respecto al resultado centralizado de referencia.

5. Conclusiones y líneas futuras

5.1. Conclusiones

En este Trabajo Fin de Grado se ha desarrollado un entorno experimental para estudiar la introducción de ruido en problemas de agregación de rankings con empates, utilizando como marco de referencia el *Optimal Bucket Order Problem* (OBOP). La principal particularidad de este problema es que la salida no es una ordenación estricta, sino un *bucket order*. Por tanto, el análisis no solo debe considerar las relaciones de precedencia entre alternativas, sino también la estructura de *buckets* obtenida, incluyendo posibles fusiones, divisiones o desplazamientos de alternativas entre grupos.

El desarrollo del trabajo ha requerido, en primer lugar, un estudio detallado de los fundamentos del problema. Antes de introducir perturbaciones o plantear la simulación federada, fue necesario analizar la agregación de rankings con empates, la formulación del OBOP, la representación mediante matrices de precedencias, la relación entre la matriz de entrada y el *bucket order* obtenido, así como los conceptos básicos de privacidad y aprendizaje federado. Esta fase permitió establecer una base sólida para diseñar las estrategias de ruido, implementar el flujo experimental y evaluar correctamente los resultados.

Como fase inicial, se estudió la resolución del OBOP mediante algoritmos heurísticos y metaheurísticos. Este análisis permitió comprender la dificultad computacional del problema, el papel de la matriz de precedencias y la forma en que se construyen soluciones en forma de *bucket order*. En particular, el estudio de estos métodos fue útil para contextualizar la necesidad de técnicas aproximadas en instancias de mayor tamaño y para entender las limitaciones asociadas al espacio de soluciones del OBOP.

A partir de este estudio inicial, se decidió utilizar una formulación exacta mediante

Programación Lineal Entera para el desarrollo experimental. Esta elección permite obtener un *bucket order* óptimo para la matriz original. Disponer de esta referencia resultaba especialmente importante, ya que el objetivo principal del trabajo era analizar cómo cambia el resultado al introducir ruido, evitando mezclar este efecto con posibles diferencias debidas al uso de una heurística aproximada.

Sobre esta base se implementó un flujo experimental para estudiar la influencia del ruido en el OBOP. El procedimiento parte de instancias reales de rankings, construye la matriz de precedencias C , resuelve el OBOP y obtiene un *bucket order* de referencia. Posteriormente, se estudiaron distintas formas de introducir perturbación sobre la información utilizada por el algoritmo. En concreto, se implementaron tres familias de estrategias: ruido directo sobre la matriz de precedencias, ruido sobre rankings individuales y ruido sobre *scores* latentes con reconstrucción posterior de la matriz.

Una vez implementadas las estrategias, se diseñó un conjunto de experimentos para evaluar su impacto mediante métricas que cuantifican la perturbación de la entrada, la concordancia con el *bucket order* de referencia, los cambios en la salida, la pérdida de calidad y la sensibilidad al ruido. Los resultados muestran que el efecto de la perturbación depende de la estrategia, su intensidad, la estructura del resultado original, la información disponible y las comparaciones concretas modificadas.

Una de las conclusiones principales es que el efecto del ruido no depende únicamente de cuánto se modifica la entrada, sino también de qué comparaciones se ven afectadas. En algunas instancias, una distancia de entrada apreciable apenas modifica el resultado final, mientras que en otras una perturbación pequeña puede alterar relaciones decisivas y provocar fusiones o fragmentaciones de *buckets*. Esto pone de manifiesto la importancia de conocer la estructura inicial de la información antes de seleccionar una estrategia de perturbación.

Los resultados no permiten identificar una estrategia claramente superior en todas las configuraciones. En este trabajo, una estrategia se considera más favorable cuando introduce una perturbación apreciable en la matriz de entrada sin deteriorar de forma significativa el *bucket order* obtenido. Sin embargo, este equilibrio depende de la instancia, del nivel de ruido y de las comparaciones concretas que se ven afectadas.

El ruido sobre matriz ofrece un control más directo sobre las relaciones modificadas, aunque puede degradar el resultado si altera comparaciones relevantes para la estructura del *bucket order*. El ruido sobre rankings individuales, en cambio, puede quedar parcialmente amortiguado durante la construcción de la matriz agregada, pero su impacto depende de las preferencias específicas que se modifiquen. Por último, las

estrategias basadas en *scores* deben interpretarse con cautela, ya que la propia reconstrucción de la matriz puede introducir cambios incluso antes de añadir ruido adicional.

El análisis experimental también ha mostrado que la evaluación del resultado no puede basarse en una única métrica. La distancia de entrada d_{entrada} cuantifica la perturbación efectiva, mientras que τ_b , la pérdida de calidad ρ , la distancia de salida d_{salida} , la sensibilidad S y el número de *buckets* permiten valorar sus efectos sobre el resultado. Por tanto, estas métricas deben analizarse conjuntamente para determinar si una mayor perturbación conserva adecuadamente el *bucket order* de referencia.

Una vez estudiado el efecto de las estrategias de perturbación en el escenario centralizado, se implementó una simulación federada con el objetivo de reducir la centralización directa de los rankings originales. Los resultados muestran que el comportamiento global del escenario federado es, en general, próximo al observado en el enfoque centralizado. Esto sugiere que la simulación federada puede ser una alternativa viable para obtener un *bucket order* final próximo al centralizado, incluso cuando las matrices locales se perturban antes de enviarse al servidor. Además, la agregación puede compensar parcialmente el ruido introducido por los clientes, aunque este efecto depende de la instancia, la técnica aplicada, el nivel de ruido y el número de clientes.

En relación con la privacidad, el trabajo debe interpretarse como un estudio experimental de perturbación. Las estrategias implementadas modifican la información utilizada por el OBOP, reducen la exposición directa de la información original y, en el escenario federado, permiten evitar que los rankings originales se compartan directamente con el servidor. Sin embargo, no proporcionan una garantía formal de privacidad diferencial. Esta limitación delimita el alcance del trabajo, centrado en analizar empíricamente el compromiso entre la perturbación introducida y la conservación de la calidad del *bucket order*.

Otra limitación importante está relacionada con la resolución exacta mediante Programación Lineal Entera. Esta elección permite obtener soluciones óptimas para las instancias estudiadas, pero limita el tamaño de los problemas que pueden abordarse con tiempos razonables. Por ello, la experimentación se ha realizado sobre instancias de tamaño reducido o moderado, suficientes para estudiar el comportamiento de las estrategias implementadas, pero no para extraer conclusiones definitivas sobre problemas de gran escala.

En definitiva, se considera que los objetivos principales del trabajo se han cumplido. Se ha estudiado el OBOP y sus métodos de resolución, se ha implementado un flujo experimental centralizado, se han diseñado e incorporado distintas estrategias de per-

turbación, se ha desarrollado una simulación federada basada en matrices locales y se ha evaluado el comportamiento del *bucket order* obtenido bajo diferentes configuraciones. El resultado final es una base experimental que permite continuar estudiando la relación entre ruido, privacidad y conservación de la calidad en problemas de agregación de rankings con empates.

5.2. Trabajos futuros

Dado el carácter exploratorio de este Trabajo Fin de Grado, existen diversas líneas futuras que podrían desarrollarse a partir de los resultados obtenidos. A continuación se recogen algunas de las más relevantes:

- Incorporar garantías formales de privacidad diferencial. Para ello sería necesario definir con precisión la sensibilidad de las consultas utilizadas, calibrar el ruido en función de un presupuesto de privacidad ϵ y estudiar cómo afecta dicho presupuesto a la calidad del *bucket order*. De esta forma, el estudio pasaría de una evaluación empírica de introducción de ruido a un marco formal de privacidad.
- Desarrollar un protocolo federado más realista. La simulación implementada reproduce el comportamiento federado dentro de un único flujo experimental. Como línea futura, podría separarse explícitamente la lógica de cliente y servidor, definiendo una comunicación real entre nodos.
- Ampliar la evaluación a instancias de mayor tamaño. Para ello, la resolución exacta mediante Programación Lineal Entera podría complementarse con algoritmos heurísticos y metaheurísticos específicos del OBOP, como BPA, variantes basadas en LIA o estrategias evolutivas. La adaptación de estos métodos al escenario federado requeriría diseñar un procedimiento iterativo, con varias rondas de actualización y agregación entre clientes y servidor, frente a la única ronda utilizada actualmente. Esto permitiría estudiar problemas de mayor escala y analizar el comportamiento de algoritmos federados iterativos.
- Abordar como una línea específica el análisis local y de equidad del escenario federado. Aunque se han calculado métricas locales con carácter complementario, el objetivo de este trabajo ha sido evaluar la conservación del resultado global respecto al escenario centralizado. En futuras ampliaciones podría estudiarse si determinados clientes o grupos quedan sistemáticamente peor representados y

desarrollar métricas o mecanismos de agregación orientados a mejorar su representación sin deteriorar excesivamente la calidad global.

5.3. Justificación de las competencias adquiridas

En este Trabajo Fin de Grado se han trabajado principalmente las siguientes competencias correspondientes a la Tecnología Específica de Computación.

[CM3] Capacidad para evaluar la complejidad computacional de un problema, conocer estrategias algorítmicas que puedan conducir a su resolución y recomendar, desarrollar e implementar aquella que garantice el mejor rendimiento de acuerdo con los requisitos establecidos.

Esta competencia se ha trabajado de forma directa durante el estudio y la implementación del OBOP. El problema presenta una elevada complejidad computacional debido al crecimiento del espacio de posibles *bucket orders*, especialmente al permitir empates entre alternativas. Para resolverlo, se ha utilizado una formulación exacta mediante Programación Lineal Entera, lo que ha permitido obtener soluciones óptimas en instancias de tamaño reducido o moderado.

Además, durante la experimentación ha sido necesario tener en cuenta las limitaciones prácticas de esta resolución exacta. La selección de instancias, el número de alternativas, las semillas y las configuraciones de ruido se han definido considerando el coste computacional del proceso. También se han estudiado alternativas heurísticas y metaheurísticas en el estado del arte, que se plantean como una vía futura para abordar instancias de mayor tamaño.

[CM4] Capacidad para conocer los fundamentos, paradigmas y técnicas propias de los sistemas inteligentes y analizar, diseñar y construir sistemas, servicios y aplicaciones informáticas que utilicen dichas técnicas en cualquier ámbito de aplicación.

Esta competencia se ha desarrollado mediante el diseño e implementación de un sistema de agregación de rankings. El trabajo ha requerido comprender cómo representar información ordinal mediante rankings, matrices de precedencias y *bucket orders*, así como construir un flujo capaz de transformar datos reales de entrada en un resultado agregado interpretable.

Además, el proyecto incorpora conceptos propios de sistemas inteligentes y tratamiento de información distribuida, como la agregación de preferencias, la introducción de ruido, la evaluación mediante métricas cuantitativas y la simulación federada. No

se ha tratado únicamente de ejecutar un algoritmo ya existente, sino de adaptar el flujo experimental para estudiar distintos escenarios, comparar estrategias y analizar su efecto sobre la calidad del resultado.

[CM7] Capacidad para conocer y desarrollar técnicas de aprendizaje computacional y diseñar e implementar aplicaciones y sistemas que las utilicen, incluyendo las dedicadas a extracción automática de información y conocimiento a partir de grandes volúmenes de datos.

Esta competencia se ha trabajado al construir un entorno que extrae información agregada a partir de colecciones de rankings. A partir de preferencias individuales, el sistema construye matrices de precedencias que resumen las relaciones por pares entre alternativas y, posteriormente, obtiene un *bucket order* que representa la información agregada de forma interpretable.

También se ha desarrollado una evaluación experimental sobre instancias reales, automatizando ejecuciones con distintas técnicas, parámetros y semillas. Los resultados se han analizado mediante métricas de similitud, pérdida, distancia y sensibilidad, lo que ha permitido transformar datos ordinales de entrada en conocimiento útil para comparar estrategias de perturbación y estudiar su comportamiento en escenarios centralizados y federados.

Bibliografía

- [1] Randy Reyna-Hernández and Alejandro Rosete. Extensión del concepto de utopía para el problema de la agregación de rankings sin empates. *TecnoLógicas*, 24(51):e1788, February 2021.
- [2] Juan A. Aledo, José A. Gámez, and Alejandro Rosete. A consensus set for the aggregation of partial rankings: The case of the optimal set of bucket orders problem. *Information Fusion*, 124:103353, December 2025.
- [3] Nir Ailon. Aggregation of partial rankings, p-ratings and top-m lists. *Algorithmica*, 57(2):284–300, 2008.
- [4] Juan A. Aledo, José A. Gámez, and Alejandro Rosete. Approaching rank aggregation problems by using evolution strategies: The case of the optimal bucket order problem. *European Journal of Operational Research*, 270(3):982–998, November 2018.
- [5] Juan A. Aledo, Concepción Domínguez, Juan de Dios Jaime-Alcántara, and Mercedes Landete. Optimizing weak orders via integer linear programming, 2025.
- [6] Jin Sima, Vishal Rana, and Olgica Milenkovic. Federated aggregation of mallows rankings: A comparative analysis of borda and lehmer coding, 2024.
- [7] Qiujun Lan, Baobao Song, Yang Li, and Gang Li. Distributed differentially private ranking aggregation. *IEEE Transactions on Computational Social Systems*, 11(1):503–513, February 2024.
- [8] Caitlin Kuhlman and Elke Rundensteiner. Rank aggregation algorithms for fair consensus. *Proceedings of the VLDB Endowment*, 13(12):2706–2719, 2020.

-
- [9] Farzad Farnoud, Behrouz Touri, and Olgica Milenkovic. Novel distance measures for vote aggregation, 2012.
 - [10] Michael Hay, Liudmila Elagina, and Gerome Miklau. *Differentially Private Rank Aggregation*, page 669–677. Society for Industrial and Applied Mathematics, 2017.
 - [11] Ravi Kumar and Sergei Vassilvitskii. Generalized distances between rankings. In *Proceedings of the 19th international conference on World wide web, WWW '10*, page 571–580. ACM, April 2010.
 - [12] Juan A. Aledo, José A. Gámez, and Alejandro Rosete. Utopia in the solution of the bucket order problem. *Decision Support Systems*, 97:69–80, May 2017.
 - [13] Juan A. Aledo, José A. Gámez, and Alejandro Rosete. A highly scalable algorithm for weak rankings aggregation. *Information Sciences*, 570:144–171, 2021.
 - [14] Cynthia Dwork and Aaron Roth. The algorithmic foundations of differential privacy. *Foundations and Trends® in Theoretical Computer Science*, 9(3-4):211–487, August 2014.
 - [15] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-Efficient Learning of Deep Networks from Decentralized Data. In Aarti Singh and Jerry Zhu, editors, *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, volume 54 of *Proceedings of Machine Learning Research*, pages 1273–1282. PMLR, 20–22 Apr 2017.
 - [16] Peter Kairouz and H. Brendan McMahan. Advances and open problems in federated learning. *Foundations and Trends® in Machine Learning*, 14(1-2):1–210, 2021.
 - [17] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with numpy. *Nature*, 585(7825):357–362, 2020.

- [18] MarkSweep. Laplace distribution pdf. Wikimedia Commons, 2005. Disponible bajo licencia CC BY-SA 3.0. Accedido el: 6 de junio de 2026.
- [19] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. *Calibrating Noise to Sensitivity in Private Data Analysis*, page 265–284. Springer Berlin Heidelberg, 2006.
- [20] M. G. Kendall. The treatment of ties in ranking problems. *Biometrika*, 33(3):239–251, 1945.
- [21] Nicholas Mattei and Toby Walsh. *PrefLib: A Library for Preferences* <http://www.preflib.org>, page 259–270. Springer Berlin Heidelberg, 2013. Preprint.
- [22] Stephen Maher, Matthias Miltenberger, João Pedro Pedroso, Daniel Rehfeldt, Robert Schwarz, and Felipe Serrano. *PySCIPOpt: Mathematical Programming in Python with the SCIP Optimization Suite*, page 301–307. Springer International Publishing, 2016.
- [23] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, Aditya Vijaykumar, Alessandro Pietro Bardelli, Alex Rothberg, Andreas Hilboll, Andreas Kloeckner, Anthony Scopatz, Antony Lee, Ariel Rokem, C. Nathan Woods, Chad Fulton, Charles Masson, Christian Häggström, Clark Fitzgerald, David A. Nicholson, David R. Hagen, Dmitrii V. Pasechnik, Emanuele Olivetti, Eric Martin, Eric Wieser, Fabrice Silva, Felix Lenders, Florian Wilhelm, G. Young, Gavin A. Price, Gert-Ludwig Ingold, Gregory E. Allen, Gregory R. Lee, Hervé Audren, Irvin Probst, Jörg P. Dietrich, Jacob Silterra, James T Webber, Janko Slavič, Joel Nothman, Johannes Buchner, Johannes Kulick, Johannes L. Schönberger, José Vinícius de Miranda Cardoso, Joscha Reimer, Joseph Harrington, Juan Luis Cano Rodríguez, Juan Nunez-Iglesias, Justin Kuczynski, Kevin Tritz, Martin Thoma, Matthew Newville, Matthias Kümmerer, Maximilian Bollingbroke, Michael Tartre, Mikhail Pak, Nathaniel J. Smith, Nikolai Nowaczyk, Nikolay Shebanov, Oleksandr Pavlyk, Per A. Brodtkorb, Perry Lee, Robert T.

McGibbon, Roman Feldbauer, Sam Lewis, Sam Tygier, Scott Sievert, Sebastiano Vigna, Stefan Peterson, Surhud More, Tadeusz Pudlik, Takuya Oshima, Thomas J. Pingel, Thomas P. Robitaille, Thomas Spura, Thouis R. Jones, Tim Cera, Tim Leslie, Tiziano Zito, Tom Krauss, Utkarsh Upadhyay, Yaroslav O. Halchenko, and Yoshiki Vázquez-Baeza. Scipy 1.0: fundamental algorithms for scientific computing in python. *Nature Methods*, 17(3):261–272, February 2020.

ANEXOS

A. Detalles de implementación

Este anexo resume la organización técnica del proyecto y las principales decisiones de implementación adoptadas para llevar a cabo los experimentos descritos en la memoria. No se pretende repetir la metodología experimental, sino indicar cómo se ha estructurado el código, qué módulos intervienen en cada fase y cómo se ejecutan los experimentos de forma reproducible.

A.1. Organización general del proyecto

El proyecto se ha implementado en Python y se ha organizado de forma modular para separar la lectura de datos, la resolución del OBOP, la introducción de ruido, la simulación federada, el cálculo de métricas y la ejecución de experimentos.

La estructura principal del repositorio es la siguiente:

```
agregacion_rankings_TFG/  
|  
|-- data/  
|   |-- 00006_skate/  
|   |-- 00035_breakfast/  
|   |-- 00068_poland_local_elections/  
|   |-- 00071_voter-autrement-in-situ/  
|  
|-- outputs/  
|   |-- baseline/  
|   |-- ruido/
```

```
|   |-- federado/
|
|-- figs/
|
|-- src/
|   |-- agregacion_ruido/
|   |-- data/
|   |-- experimentos/
|   |-- federado/
|   |-- metricas/
|   |-- obop/
|   |-- run_baseline.py
|   |-- run_ruido_experiments.py
|   |-- run_federado_experiments.py
|
|-- requirements.txt
|-- README.md
```

La carpeta **data/** contiene las instancias de PrefLib utilizadas en los experimentos. La carpeta **src/** contiene el código fuente. La carpeta **outputs/** almacena los ficheros CSV generados por las ejecuciones, mientras que **figs/** contiene las figuras utilizadas para resumir parte de los resultados.

A.2. Módulos principales

La Tabla [A.1](#) resume los módulos principales del proyecto y su función dentro del pipeline experimental.

Esta separación permite que la lógica del problema, las estrategias de perturbación y los scripts de ejecución se mantengan independientes. De este modo, el código puede ampliarse añadiendo nuevas estrategias de ruido, nuevas métricas o variantes del procedimiento federado sin modificar todo el pipeline.

Módulo	Función principal
<code>data/preflib_to_C.py</code>	Lectura de ficheros PrefLib, extracción de rankings y construcción de la matriz de precedencias C .
<code>obop/obop_ilp.py</code>	Resolución exacta del OBOP mediante Programación Lineal Entera. Incluye la versión estándar y la versión ponderada utilizada en la simulación federada.
<code>obop/bucket_order.py</code>	Reconstrucción del <i>bucket order</i> a partir de la solución binaria devuelta por el modelo ILP.
<code>agregacion_ruido/</code>	Implementación de las tres familias de perturbación: ruido sobre matriz, ruido sobre rankings individuales y ruido sobre <i>scores</i> latentes.
<code>federado/</code>	Implementación de la simulación federada: construcción de matrices locales, rol de cliente, rol de servidor y agregación de matrices locales perturbadas.
<code>metricas/</code>	Cálculo de las métricas utilizadas para comparar <i>bucket orders</i> y matrices antes y después de introducir ruido.
<code>experimentos/</code>	Lógica experimental común para <i>baseline</i> , experimentos centralizados con ruido y simulación federada.
<code>run_*.py</code>	Scripts ejecutables desde terminal para lanzar los experimentos y guardar los resultados en CSV.

Tabla A.1: Módulos principales de la implementación.

A.3. Lectura de datos y construcción de la matriz de precedencias

La lectura de datos se realiza a partir de ficheros PrefLib con extensiones `.soc`, `.soi`, `.toc` y `.toi`. Estos formatos permiten trabajar con rankings completos, rankings incompletos, rankings estrictos y rankings con empates.

Cada ranking se representa internamente como una lista de *buckets*. Las alternativas situadas dentro de un mismo *bucket* se consideran empatadas, mientras que el orden entre *buckets* representa la relación de precedencia.

En los ficheros PrefLib, un mismo ranking puede aparecer asociado a una multiplicidad, que indica cuántos votantes han expresado exactamente esa misma ordenación. Por este motivo, durante la lectura no se trata cada línea como un único ranking, sino que se conserva su multiplicidad y se utiliza como peso en la construcción de la matriz de precedencias.

A partir de la colección de rankings se construye la matriz de precedencias C . Para cada par de alternativas (u, v) , se contabiliza la proporción ponderada de rankings en

los que u precede a v , teniendo en cuenta las multiplicidades de cada ordenación. Si ambas alternativas aparecen empatadas, se añade una contribución de 0.5. En rankings incompletos, si alguna de las dos alternativas no aparece, ese ranking no aporta información sobre el par.

La matriz construida cumple las propiedades utilizadas por el OBOP:

$$C(u, u) = 0.5, \quad C(u, v) + C(v, u) = 1.$$

Estas condiciones se comprueban mediante funciones de validación antes de resolver el problema.

A.4. Resolución del OBOP

La resolución del OBOP se implementa en el módulo `obop/obop_ilp.py`. El modelo utiliza variables binarias x_{rs} , que indican si la alternativa r precede o está empatada con la alternativa s . A partir de estas variables se representa la matriz asociada al *bucket order* candidato y se minimiza la distancia L_1 respecto a la matriz de precedencias de entrada.

La implementación utiliza `PySCIP0pt` como interfaz para resolver el modelo de Programación Lineal Entera [22]. Una vez obtenida la solución, el módulo `bucket_order.py` reconstruye el *bucket order* agrupando las alternativas que aparecen mutuamente empatadas y ordenando los *buckets* según las relaciones de precedencia obtenidas.

Además de la versión estándar, se incluye una versión ponderada del OBOP. Esta variante se utiliza en la simulación federada para resolver el problema local de cada cliente considerando únicamente los pares observados. Para ello, se emplea una máscara P_i , donde $P_i(u, v) = 1$ si el par ha sido observado por el cliente i , y $P_i(u, v) = 0$ en caso contrario.

A.5. Estrategias de introducción de ruido

Las estrategias de ruido se implementan en la carpeta `agregacion_ruido/`. Se han considerado tres familias:

- **Ruido sobre matriz:** modifica directamente las entradas de la matriz de precedencias C .

- **Ruido sobre rankings:** aplica operaciones discretas sobre los rankings individuales antes de reconstruir la matriz perturbada.
- **Ruido sobre scores latentes:** estima una puntuación para cada alternativa, perturba dichas puntuaciones y reconstruye una nueva matriz de precedencias.

En todos los casos, la salida de la estrategia es una matriz perturbada $\tilde{C}$, que se utiliza como entrada del OBOP para obtener un nuevo *bucket order*. La semilla aleatoria se fija desde los scripts de ejecución para que los experimentos puedan reproducirse.

A.6. Implementación de la simulación federada

La simulación federada descrita metodológicamente en la Sección 3.3 se implementa principalmente en la carpeta `src/federado/`. Esta parte del código no corresponde a un despliegue real distribuido, sino a una simulación controlada dentro del mismo entorno Python. Su objetivo es reproducir el flujo cliente-servidor definido en la metodología, manteniendo separados los datos locales de cada cliente y la agregación realizada por el servidor.

La implementación se reparte en tres módulos principales:

- `agregacion_federada.py`: contiene las funciones auxiliares para dividir los rankings entre clientes, construir las matrices locales W_i , M_i y C_i , perturbar las matrices locales observadas y agregar las matrices recibidas por el servidor.
- `cliente.py`: define la clase `ClienteFederado`. Cada cliente almacena sus rankings locales, construye su matriz local, resuelve su *bucket order* local para el análisis complementario, aplica ruido sobre los pares observados y genera el mensaje que se envía al servidor.
- `servidor.py`: define la clase `ServidorFederado`. El servidor recibe los mensajes de los clientes, agrega las matrices locales perturbadas $\tilde{C}_i$ usando M_i como peso por par, resuelve el OBOP sobre la matriz federada y devuelve el *bucket order* federado.

Para complementar la descripción anterior, la Figura A.1 muestra el diagrama de clases de la simulación federada. Dado que la mayor parte del proyecto se ha implementado mediante módulos funcionales, el diagrama se centra únicamente en la parte orientada a objetos, donde se modelan explícitamente los roles de cliente y servidor y el mensaje intercambiado entre ambos.

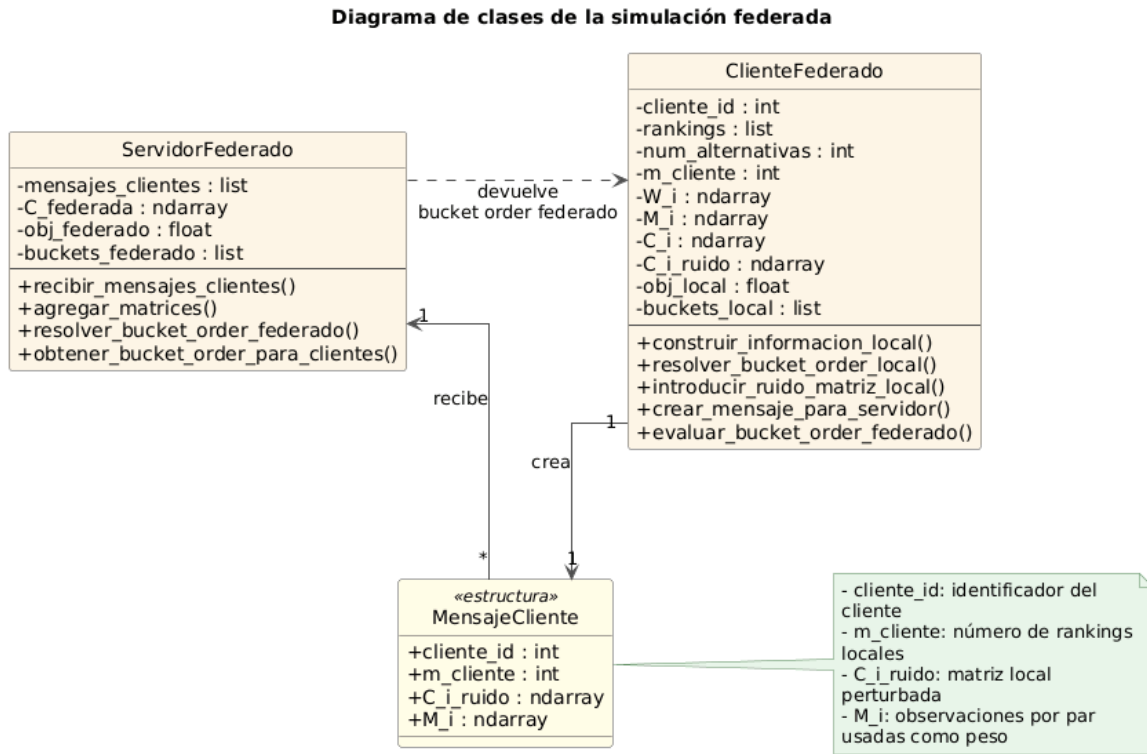


Figura A.1: Diagrama de clases de la simulación federada.

A.7. Métricas y resultados

Las métricas se implementan en la carpeta **metricas/**. En los experimentos centralizados se comparan el *bucket order* original y el *bucket order* obtenido tras introducir ruido. Para ello se calculan métricas como el coeficiente de correlación de Kendall τ_b , la pérdida de calidad ρ , la distancia de entrada d_{entrada} , la distancia de salida d_{salida} y la sensibilidad S .

En la simulación federada se registran métricas globales y locales. Las métricas globales comparan el *bucket order* federado con el *bucket order* centralizado. Las métricas locales evalúan cómo se ajusta el *bucket order* federado a la información disponible en cada cliente.

Los resultados se almacenan en ficheros CSV dentro de **outputs/**. Se generan tres tipos de resultados:

- **outputs/baseline/**: resultados del OBOP centralizado sin ruido.
- **outputs/ruido/**: resultados de los experimentos centralizados con ruido.

- `outputs/federado/`: resultados de la simulación federada.

A.8. Ejecución de los experimentos

Los experimentos se lanzan desde la raíz del proyecto mediante tres scripts principales. El *baseline* centralizado se ejecuta con:

```
python src/run_baseline.py <fichero_preflib>
```

Los experimentos centralizados con ruido se ejecutan indicando el método de perturbación, la técnica, los valores del parámetro b y las semillas:

```
python src/run_ruido_experiments.py <fichero_preflib> \  
--metodo matriz \  
--tecnica cerca_empate \  
--b 0,0.05,0.10 \  
--seeds 0,1,2
```

La simulación federada se ejecuta indicando la técnica de perturbación local, los valores de b , el número de clientes y las semillas:

```
python src/run_federado_experiments.py <fichero_preflib> \  
--tecnica cerca_empate \  
--b 0,0.05,0.10 \  
--num_clientes 2,3 \  
--seeds 0,1,2
```

En todos los casos, cada ejecución trabaja sobre una instancia concreta de Pre-`fLib`. Esta decisión simplifica la trazabilidad de los resultados y permite identificar claramente qué configuración experimental ha generado cada fichero CSV.

A.9. Dependencias

Las dependencias principales del proyecto se recogen en el fichero `requirements.txt`. Las librerías utilizadas son:

- `numpy`, para el manejo de matrices y operaciones numéricas.

-
- `scipy`, para funciones estadísticas utilizadas en algunas métricas y en la reconstrucción mediante *scores*.
 - `pyscipopt`, para resolver el modelo ILP del OBOP.

El desarrollo se ha realizado manteniendo el código separado por responsabilidades, de forma que los experimentos puedan reproducirse desde los scripts principales y los resultados puedan revisarse posteriormente a partir de los ficheros CSV generados.

B. Resultados complementarios de los experimentos con ruido

En este anexo se recogen los resultados individuales que complementan el análisis presentado en el Capítulo 4. Las tablas mantienen las semillas 0, 1 y 2 y muestran las métricas empleadas en el estudio: distancia de entrada, distancia de salida, sensibilidad, coeficiente de Kendall, pérdida de calidad y número de *buckets*.

B.1. Instancia 00035-00000006.soc

La Tabla B.1 recoge las ejecuciones de ruido sobre rankings que no se incluyen en el cuerpo principal.

b	Semilla	d_{entrada}	d_{salida}	S	τ_b	ρ	Buckets
0.10	0	0.011	0.000	0.000	1.000	0.000	2
	1	0.006	0.000	0.000	1.000	0.000	2
	2	0.003	0.000	0.000	1.000	0.000	2
0.50	0	0.028	0.124	4.457	0.000	0.006	1
	1	0.036	0.124	3.478	0.000	0.006	1
	2	0.040	0.067	1.670	0.681	0.002	2
1.00	0	0.077	0.124	1.611	0.000	0.006	1
	1	0.067	0.124	1.851	0.000	0.006	1
	2	0.070	0.124	1.761	0.000	0.006	1

Tabla B.1: Ejecuciones de la estrategia de introducción de ruido sobre rankings para la instancia 00035-00000006.soc.

B.2. Instancia 00068-00000301.soi

La Tabla B.2 presenta los resultados de las tres técnicas de perturbación sobre matriz.

Técnica	b	Semilla	d_{entrada}	d_{salida}	S	τ_b	ρ	Buckets
todos	0.05	0	0.048	0.071	1.495	0.831	0.005	4
		1	0.037	0.000	0.000	1.000	0.000	5
		2	0.037	0.000	0.000	1.000	0.000	5
	0.10	0	0.095	0.071	0.749	0.831	0.005	4
		1	0.075	0.000	0.000	1.000	0.000	5
		2	0.073	0.000	0.000	1.000	0.000	5
	0.20	0	0.176	0.248	1.409	0.461	0.048	2
		1	0.142	0.000	0.000	1.000	0.000	5
		2	0.139	0.100	0.718	0.830	0.049	7
aleatoria	0.20	0	0.040	0.119	2.952	0.756	0.002	4
		1	0.028	0.000	0.000	1.000	0.000	5
		2	0.041	0.000	0.000	1.000	0.000	5
	0.50	0	0.088	0.124	1.407	0.749	0.006	5
		1	0.060	0.057	0.945	0.906	0.016	6
		2	0.065	0.000	0.000	1.000	0.000	5
	1.00	0	0.127	0.157	1.233	0.690	0.024	4
		1	0.095	0.095	0.999	0.805	0.035	7
		2	0.084	0.010	0.113	0.985	0.006	6
cerca_empate	0.10	0	0.060	0.000	0.000	1.000	0.000	5
		1	0.051	0.000	0.000	1.000	0.000	5
		2	0.045	0.000	0.000	1.000	0.000	5
	0.50	0	0.177	0.257	1.457	0.547	0.133	10
		1	0.174	0.214	1.230	0.680	0.129	10
		2	0.162	0.214	1.319	0.680	0.119	10
	1.00	0	0.222	0.181	0.813	0.780	0.143	13
		1	0.212	0.162	0.765	0.812	0.152	11
		2	0.209	0.214	1.024	0.686	0.129	12

Tabla B.2: Ejecuciones de la estrategia de introducción de ruido sobre matriz para la instancia 00068-00000301.soi.

La Tabla B.3 recoge las ejecuciones de las reconstrucciones `logistic` y `probit`.

Técnica	b	Semilla	d_{entrada}	d_{salida}	S	τ_b	ρ	Buckets
logistic	0.00	–	0.209	0.076	0.364	0.871	0.025	3
		0	0.209	0.129	0.615	0.736	0.011	3
	0.20	1	0.217	0.162	0.747	0.704	0.054	3
		2	0.211	0.076	0.361	0.871	0.025	3
		0	0.262	0.176	0.673	0.657	0.087	4
	0.50	1	0.237	0.214	0.904	0.609	0.087	3
		2	0.223	0.138	0.619	0.753	0.049	3
probit	0.00	–	0.225	0.152	0.677	0.780	0.092	4
		0	0.227	0.129	0.567	0.782	0.087	4
	0.20	1	0.234	0.190	0.813	0.711	0.102	4
		2	0.228	0.152	0.667	0.780	0.092	4
		0	0.293	0.233	0.797	0.527	0.144	5
	0.50	1	0.255	0.157	0.616	0.730	0.087	5
		2	0.247	0.214	0.869	0.622	0.135	5

Tabla B.3: Ejecuciones de la estrategia de introducción de ruido sobre *scores* para la instancia 00068-00000301.soi.

B.3. Instancia 00071-00000033.toi

La Tabla B.4 presenta los resultados complementarios del ruido introducido sobre los rankings individuales.

b	Semilla	d_{entrada}	d_{salida}	S	τ_b	ρ	Buckets
0.10	0	0.007	0.000	0.000	1.000	0.000	2
	1	0.005	0.000	0.000	1.000	0.000	2
	2	0.005	0.000	0.000	1.000	0.000	2
0.50	0	0.033	0.218	6.573	0.000	0.028	1
	1	0.030	0.218	7.234	0.000	0.028	1
	2	0.030	0.218	7.245	0.000	0.028	1
1.00	0	0.065	0.218	3.356	0.000	0.028	1
	1	0.065	0.218	3.344	0.000	0.028	1
	2	0.057	0.218	3.815	0.000	0.028	1

Tabla B.4: Ejecuciones de la estrategia de introducción de ruido sobre rankings para la instancia 00071-00000033.toi.

B.4. Métricas locales complementarias de la simulación federada

Como complemento al análisis global, esta sección presenta las métricas calculadas sobre la información local de los clientes. Estos resultados tienen carácter descriptivo y no se utilizan como criterio principal para evaluar la simulación federada, cuyo objetivo es comparar el *bucket order* federado con el resultado centralizado de referencia.

En la Tabla B.5, $\overline{d_{\text{entrada},i}}$ representa la perturbación media introducida localmente en las matrices de los clientes y la métrica $\overline{\rho_i}$ recoge la pérdida local media.

Instancia	Técnica	K	$\overline{d_{\text{entrada},i}}$	$\overline{\rho_i}$
00006-00000040	aleatoria	2	0.053	0.013
	cerca_empate	2	0.017	0.012
	todos	2	0.071	0.012
00035-00000006	aleatoria	4	0.062	0.030
	cerca_empate	4	0.083	0.027
	todos	4	0.106	0.029
00068-00000301	aleatoria	2	0.041	0.111
	cerca_empate	2	0.011	0.084
	todos	2	0.054	0.158
00071-00000033	aleatoria	4	0.064	0.007
	cerca_empate	4	0.081	0.000
	todos	4	0.106	0.003

Tabla B.5: Resumen de las métricas locales en la simulación federada.

Los resultados muestran que las métricas locales varían principalmente según la instancia. En particular, 00068-00000301 presenta valores más altos de pérdida local media, lo que indica una mayor diferencia entre algunas matrices locales y el resultado global federado.

C. Recursos software y hardware

C.1. Recursos software

En el desarrollo de este Trabajo Fin de Grado se han utilizado diversas herramientas y librerías de software, tanto para la implementación del entorno experimental como para la redacción de la memoria. A continuación, se detallan los principales recursos empleados:

- **Python**: lenguaje principal utilizado para la implementación del entorno experimental.
- **NumPy**: manejo de matrices, operaciones numéricas y generación de ruido aleatorio.
- **SciPy**: cálculo de funciones estadísticas, incluyendo la correlación Kendall τ_b y funciones utilizadas en la estrategia basada en *scores* latentes [23].
- **PySCIPOpt**: interfaz de Python utilizada para formular y resolver el modelo de Programación Lineal Entera.
- **SCIP**: solver de optimización empleado para resolver las instancias del OBOP.
- **PrefLib**: fuente de los conjuntos de datos ordinales empleados en los experimentos.
- **Biblioteca estándar de Python**: gestión de argumentos, rutas, ficheros, tiempos de ejecución y almacenamiento de resultados.
- **L^AT_EX**: redacción y maquetación de la memoria del Trabajo Fin de Grado.
- **Visual Studio Code**: entorno de desarrollo utilizado para el desarrollo del código y la edición de la memoria.

-
- **GitHub**: plataforma utilizada para el control de versiones del proyecto.

C.2. Recursos hardware

Todas las ejecuciones se realizaron en el mismo equipo, con el fin de que los tiempos de resolución fueran comparables entre configuraciones. Las características principales del sistema utilizado se muestran en la Tabla C.1.

Componente	Especificación
Procesador	11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40 GHz
Memoria RAM	16 GB
Tarjeta gráfica	Intel(R) Iris(R) Xe Graphics
Sistema operativo	Windows 11, 64 bits

Tabla C.1: Recursos hardware del sistema utilizado para las ejecuciones experimentales.